Published: 18 June 2025

RESEARCH-ARTICLE

Accelerating Graph Indexing for ANNS on Modern CPUs

MENGZHAO WANG, Zhejiang University, Hangzhou, Zhejiang, China

HAOTIAN WU, Zhejiang University, Hangzhou, Zhejiang, China

XIANGYU KE, Zhejiang University, Hangzhou, Zhejiang, China

YUNJUN GAO, Zhejiang University, Hangzhou, Zhejiang, China

YIFAN ZHU, Zhejiang University, Hangzhou, Zhejiang, China

WENCHAO ZHOU, Alibaba Group Holding Limited, Hangzhou, Zhejiang, China

Proceedings of the ACM on Management of Data, Volume 3, Issue 3 (June 2025)
https://doi.org/10.1145/3725260
EISSN: 2836-6573

Accelerating Graph Indexing for ANNS on Modern CPUs

MENGZHAO WANG, Zhejiang University, China
HAOTIAN WU, Zhejiang University, China
XIANGYU KE, Zhejiang University, China
YUNJUN GAO, Zhejiang University, China
YIFAN ZHU, Zhejiang University, China
WENCHAO ZHOU, Alibaba Group, China

In high-dimensional vector spaces, Approximate Nearest Neighbor Search (ANNS) is a key component in database and artificial intelligence infrastructures. Graph-based ANNS methods, particularly HNSW, have emerged as leading solutions, offering an impressive trade-off between search efficiency and accuracy. Many vector databases utilize graph indexes as their core algorithms, benefiting from various optimizations to enhance search performance. However, the high indexing time associated with graph algorithms poses a significant challenge, especially given the increasing volume of data, query processing complexity, and dynamic index maintenance demand. This has rendered indexing time a critical performance metric for users.

In this paper, we comprehensively analyze the underlying causes of the low graph indexing efficiency on modern CPUs, identifying that distance computation dominates indexing time, primarily due to high memory access latency and suboptimal arithmetic operation efficiency. We demonstrate that distance comparisons during index construction can be effectively performed using compact vector codes at an appropriate compression error. Drawing from insights gained through integrating existing compact coding methods in the graph indexing process, we propose a novel compact coding strategy, named `Flash`, designed explicitly for graph indexing and optimized for modern CPU architectures. By minimizing random memory accesses and maximizing the utilization of SIMD (Single Instruction, Multiple Data) instructions, `Flash` significantly enhances cache hit rates and arithmetic operations. Extensive experiments conducted on eight real-world datasets, ranging from ten million to one billion vectors, exhibit that `Flash` achieves a speedup of 10.4× to 22.9× in index construction efficiency, while maintaining or improving search performance.

CCS Concepts: • **Information systems → Data compression**; **Search index compression**.

Additional Key Words and Phrases: Approximate Nearest Neighbor Search; Graph-based Vector Index; Vector Compression

ACM Reference Format:
Mengzhao Wang, Haotian Wu, Xiangyu Ke, Yunjun Gao, Yifan Zhu, and Wenchao Zhou. 2025. Accelerating Graph Indexing for ANNS on Modern CPUs. *Proc. ACM Manag. Data* 3, 3 (SIGMOD), Article 123 (June 2025), 29 pages. https://doi.org/10.1145/3725260

Authors' Contact Information: Mengzhao Wang, Zhejiang University, China, wmzssy@zju.edu.cn; Haotian Wu, Zhejiang University, China, haotian.wu@zju.edu.cn; Xiangyu Ke, Zhejiang University, China, xiangyu.ke@zju.edu.cn; Yunjun Gao, Zhejiang University, China, gaoyj@zju.edu.cn; Yifan Zhu, Zhejiang University, China, xtf_z@zju.edu.cn; Wenchao Zhou, Alibaba Group, China, zwc231487@alibaba-inc.com.

ACM 2836-6573/2025/6-ART123
https://doi.org/10.1145/3725260

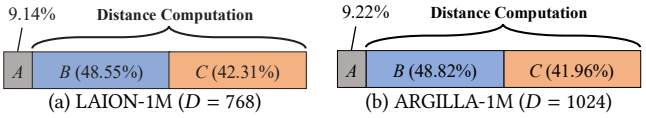
1 Introduction

Approximate Nearest Neighbor Search (ANNS) in high-dimensional spaces has become integral due to the advent of deep learning embedding techniques for managing unstructured data [51, 88, 100, 104]. This spans a variety of applications, such as recommendation systems [41, 58], information retrieval [54, 106], and vector databases [56, 83]. Owing to the "curse of dimensionality" and the ever-increasing volume of data, ANNS seeks to optimize both speed and accuracy, making it more practical compared to the exact solutions, which can be prohibitively time-consuming. Given a query vector, ANNS efficiently finds its k nearest vectors in a vector dataset following a specific distance metric. Current studies explore four index types for ANNS: Tree-based [32, 76], Hash-based [60, 127], Quantization-based [30, 67], and Graph-based methods [49, 75]. Among these, graph-based algorithms like HNSW [78] have demonstrated superior performance, establishing themselves as the industry standard [11, 45, 46] and attracting substantial academic interest [50, 71, 91]. For example, HNSW underpins various ANNS services or vector databases, including PostgreSQL [114], ElasticSearch [11], Milvus [100], and Pinecone [18]. Recent publications from the data management community [12–14] highlight the focus on ANNS research regarding HNSW[1].

Although numerous studies [39, 42, 50, 71, 75, 89, 116] have explored optimizations for search performance, research on improving index construction efficiency remains limited[2], which is increasingly critical in real-world applications [111]. First, the *vast volume of data* poses significant challenges for index construction. For instance, building an HNSW index on tens of millions of vectors typically requires about *10 hours* [45, 75], billion-scale datasets may extend construction time to *five days*, even with relaxed construction parameters [66]. Second, complex query processing, such as hybrid search [53, 102], adds constraints on ANNS, *complicating the construction process* and increasing indexing times [128]. For example, constructing a specialized HNSW index for attribute-constrained ANNS takes 33× longer than a standard index [85]. More importantly, *continuous data and embedding model updates* [25, 27, 61, 72, 94] necessitate a periodic reconstruction process based on the LSM-Tree framework [5, 37, 84, 100, 108, 111]. While some approaches aim to avoid rebuilding [93], they are often unsuitable for embedding model updates [25, 90] and can result in diminished search performance [5], with accuracy dropping from 0.95 to 0.88 after 20 update cycles on HNSW [93]. Thus, reconstruction efficiency has become a bottleneck in modern vector databases [84], as it ***directly impacts the ability to rapidly release new services with up-to-date data and high search performance*** [93, 108]. In business scenarios, index rebuilding often occurs overnight during low user activity [108], constrained to a few hours. Serving approximately 100M vectors on a single node, the HNSW build time frequently exceeds 12 hours, failing to meet the requirement.

To enhance the index construction efficiency, employing specialized hardware like GPUs is a straightforward approach [92, 98]. By leveraging GPUs' parallel computing capabilities, recent studies have parallelized distance computations and data structure maintenance for the graph indexing process on *million-scale* datasets [82, 115, 125], achieving an order of magnitude speedup over single-thread CPU implementations [115]. However, challenges such as high costs, memory constraints, and additional engineering efforts limit their practical applications [124]. For instance, even high-end GPUs like the NVIDIA A100, with tens of gigabytes of memory, struggle to accommodate large vector datasets requiring hundreds of gigabytes. As noted by Stonebraker in his recent survey [28], "*If data does not fit in GPU memory, query execution bottlenecks on loading data into the device, significantly diminishing parallelization benefits.*" Consequently, CPU-based

[1]Unless otherwise specified, we illustrate our framework design using this representative graph-based index.
[2]Since multi-thread graph indexing on CPUs is a standard practice, we present all discussed graph algorithms in their multi-thread versions by default.



(a) LAION-1M (D = 768) (b) ARGILLA-1M (D = 1024)

Fig. 1. Profiling of HNSW indexing time. Distance computation constitutes the majority of the indexing time and consists of two components: memory accesses (B) and arithmetic operations (C). Other tasks (A), such as data structure maintenance, account for a minor portion.

deployment remains the prevalent choice in practical scenarios [42, 46, 100, 114], providing a cost-effective, memory-sufficient, and easy-to-use solution. This motivates us to explore optimization opportunities to improve the index construction efficiency on modern CPUs.

In this paper, we conduct an in-depth analysis of the HNSW construction process, identifying the root causes of inefficiency on modern CPUs. Figure 1 illustrates HNSW indexing time profiled using the *perf* tool [26] on two real-world embedding datasets [16, 24]. Results indicate that distance computation constitutes over 90% of the total construction time, marking it as the major bottleneck, while memory accesses and arithmetic operations contribute similarly. The lack of spatial locality in the HNSW index necessitates random memory accesses to fetch vectors for almost every distance computation. Building HNSW on a dataset of size n requires $O(n \log(n))$ distance calculations [78], leading to frequent cache misses as target data is often uncached. As a result, the memory controller frequently loads data from main memory, resulting in high access latency. Additionally, current vector data typically consists of high-dimensional floating-point numbers, occupying $4 \cdot D$ bytes for dimension D (e.g., 3KB for D = 768). However, Single Instruction, Multiple Data (SIMD) registers are restricted to hundreds of bits (e.g., 128 bits for the SSE instruction set) [21], much smaller than vector sizes. Thus, each distance computation requires hundreds of memory reads to load vector segments into a register, reducing the efficiency of SIMD operations. Notably, all distances are calculated solely for comparison during HNSW construction (refer to Section **2.2**). Recent studies suggest that exact distances are unnecessary for comparison [50, 107, 113].

To address these issues, we optimize the HNSW construction process by reducing random memory accesses and effectively utilizing SIMD instructions. The theoretical analysis demonstrates that distance comparisons during index construction can be performed effectively using compact vector codes at an appropriate compression error. We initially implement mainstream vector compression methods—Product Quantization (PQ) [67], Scalar Quantization (SQ) [29], and Principal Component Analysis (PCA) [69]—into the HNSW construction process. However, these methods yield limited improvements in indexing efficiency or degrade search performance, as they do not align with HNSW's construction characteristics. Further observations show that they fail to reduce excessive random memory accesses and do not leverage SIMD advantages. This motivates us to develop a speci**f**ic compact coding strategy, a**l**ongside memory l**a**yout optimization for the con**s**truction process of **H**NSW, named `Flash`. To maximize SIMD instruction utilization, `Flash` is designed to accommodate SIMD register features, enabling concurrent execution of distance computations within the SIMD register. To minimize random memory accesses, `Flash` is effectively organized to match data access and register load patterns. Ultimately, `Flash` avoids numerous random memory accesses and enhances SIMD usage, achieving an order of magnitude speedup in construction efficiency while maintaining or improving search performance. To the best of our knowledge, this is the first work to reach such significant speedup on modern CPUs.

To sum up, this paper makes the following contributions.

- We deeply analyze the construction process of HNSW and identify the root causes of low construction efficiency on modern CPUs. Specifically, we find that distance computation suffers

from high memory access latency and low arithmetic operation efficiency, which accounts for over 90% of indexing time.
- We propose a novel compact coding strategy, `Flash`, specifically designed for HNSW construction, optimizing index layout for efficient memory accesses and SIMD utilization. This approach improves cache hits and register-resident computation.
- We conduct extensive evaluations on real-world datasets, ranging from ten million to billion-scale, demonstrating that construction efficiency can be improved by over an order of magnitude, while search performance remains the same or even improves.
- We summarize three notable findings from our research: (1) a compact coding method that significantly enhances search performance may not be suitable for index construction; (2) reducing more dimensions may bring higher accuracy; and (3) encoding vectors and distances with a tiny amount of bits to align with hardware constraints may yield substantial benefits.

The paper is organized as follows: Section 2 reviews related work and conducts a problem study on index construction of HNSW. Section 3 introduces the proposed novel compact coding strategy alongside the memory layout optimization for HNSW construction. Section 4 provides experimental results, while some notable findings and conclusion are outlined in Sections 5 and 6, respectively.

2 Background and Motivation

In this section, we first review current graph-based ANNS algorithms to ascertain the leadership of HNSW, then outline search and indexing optimizations based on HNSW. Next, we provide a detailed analysis of the HNSW construction process to identify the root causes of inefficiency on modern CPUs.

2.1 Related Work

2.1.1 ***Graph-based ANNS Algorithms.*** Graph-based ANNS algorithms have emerged as the leading approach for balancing search accuracy and efficiency in high-dimensional spaces [38, 48, 66, 73, 88, 104]. These methods construct a graph index where vertices represent database vectors, and directed edges signify neighbor relationships. During search, a greedy algorithm navigates the graph, visiting a vertex's neighbors and selecting the one closest to the query, continuing until no closer neighbor is found, yielding the final result. Current methods share similar index structures and greedy search strategies but differ in their edge selection strategies during graph construction [104]. For example, HNSW [78] and NSG [49] use the Monotonic Relative Neighborhood Graph (MRNG) strategy, while Vamana [66] and τ-MG [88] incorporate additional parameters. HCNNG [81] employs the Minimum Spanning Tree (MST) approach, while KGraph [44] and NGT [64] implement the K-Nearest Neighbor Graph (KNNG) paradigm. Despite these variations, state-of-the-art methods like HNSW, NSG, Vamana, and τ-MG adhere to a similar construction framework, consisting of Candidate Acquisition (CA) and Neighbor Selection (NS) stages [49, 88]. Our research identifies the bottleneck in the CA and NS steps, where distance calculations dominate indexing time (see Section 2.2). Consequently, improving the CA and NS steps will accelerate the indexing process in all these graph algorithms. Notably, HNSW stands out for its robust optimizations and versatile features [39, 74, 128], including native *add* support. Numerous studies have enhanced HNSW from various perspectives (we review them in the following paragraphs), solidifying its role as a mainstream method in both industry and academia.

2.1.2 ***Search Optimizations on HNSW.*** Search optimizations for HNSW focus on several key components. Some aim to optimize entry point acquisition by leveraging additional quantization or hashing structures to start closer to the query [75, 126]. Others optimize neighbor visits by

learning global [35] or topological [80] information, enhancing data locality [42], and partitioning neighbor areas [110]. These strategies improve accuracy and efficiency by identifying relevant neighbors and skipping irrelevant ones. Some approaches reduce distance computation complexity using approximate calculations [39, 50, 77, 113], enhancing search efficiency with minimal accuracy loss. Additionally, optimizations refine termination conditions through adaptive decision-making [71, 74, 112] and relaxed monotonicity [121]. Specialized hardware like GPUs [125], FPGAs [68, 87, 117], Compute Express Link (CXL) [65], Non-Volatile Memory (NVM) [91], NVMe SSDs [103], and SmartSSDs [97] accelerate distance computation and optimize the HNSW index layout, achieving superior performance through software-hardware collaboration. To support complex query processing, some approaches integrate structured predicates [85, 101] or range attributes [128] into HNSW, enabling hybrid searches of vectors and attributes. These optimizations enhance HNSW's performance and versatility, meeting diverse real-world requirements, though most increase indexing time compared to the original HNSW.

2.1.3 ***Indexing Optimizations of HNSW.*** Despite HNSW's superior search performance, its low index construction efficiency remains a major bottleneck, a challenge shared by other graph-based methods [33, 48, 49, 66, 75, 126]. As demands for dynamic updates [93, 111], large-scale data [40, 66], and complex queries [85, 128] grow (see Section 1), optimizing HNSW construction becomes increasingly important. Current research targets two areas: algorithm optimization and hardware acceleration. Algorithm optimization efforts have redesigned the construction pipeline, either fully [49, 88] or partially [75], but these attempts offer only marginal improvements [48, 49, 126]. Moreover, these optimizations substantially modify the algorithmic process of HNSW, requiring code refactoring that may disrupt many engineering optimizations integrated over the years [2–4]. Some HNSW features, like native *add* support, are weakened [75] or even discarded [49]. On the other hand, GPU-accelerated methods parallelize distance computations during HNSW construction [55, 82, 98]. These methods re-implement HNSW to align with GPU architecture, achieving an order of magnitude speedup compared to single-thread CPU-based construction [115]. However, GPU-specific limitations reduce their practicality in real-world scenarios [124] (see Section 1). As CPU-based HNSW remains prevalent in many industrial applications [42, 46, 100, 114], accelerating HNSW construction on modern CPUs is crucial. Note that parallel multi-thread implementations of graph indexing on CPUs are simple, efficient, and widely adopted by current graph indexes (e.g., HNSW, NSG [49], τ-MG [88]). These implementations are orthogonal to our work, as our techniques build upon multi-thread graph indexing.

2.1.4 ***Distributed Deployment of HNSW.*** Distributed deployment accelerates HNSW construction by partitioning large datasets into multiple shards, each containing tens of millions of vectors [43, 45, 56, 63]. This enables concurrent index construction across nodes, enhancing efficiency for large-scale datasets [45]. Query processing can be optimized by an inter-shard coordinator [56], and advanced data fragmentation strategies can assign queries to relevant shards [34, 120]. At the system level, distributed deployment ensures scaling, load balancing, and fault tolerance [103], but introduces challenges like high communication overhead [79, 100]. Notably, distributed deployment is orthogonal to our research, as we focus on efficient index building within a shard, which can be directly integrated into existing distributed systems.

2.2 Problem Analysis

Notations. In the HNSW index, each vertex corresponds to a unique vector, denoted by bold lowercase letters (e.g., $\boldsymbol{x}$, $\boldsymbol{y}$). We use the Euclidean distance $\delta(\boldsymbol{x}, \boldsymbol{y})$ as the distance metric for quantifying similarity, where a smaller $\delta(\boldsymbol{x}, \boldsymbol{y})$ indicates greater similarity [49, 88, 104, 126]. Bold

Algorithm 1: Index Construction of HNSW

Input: a vector dataset $\boldsymbol{S}$, hyper-parameters C and R ($R \leq C$)
Output: HNSW index built on $\boldsymbol{S}$

1 $\boldsymbol{V} \leftarrow \emptyset$, $\boldsymbol{E} \leftarrow \emptyset$; ▷ start from a empty graph
2 **for** *each* $\boldsymbol{x}$ *in* $\boldsymbol{S}$ **do** ▷ insert all vectors in $\boldsymbol{S}$
3 $l_{max} \leftarrow \boldsymbol{x}$'s max layer; ▷ exponential decaying distribution
4 **for** *each* $l \in \{l_{max}, \cdots, 0\}$ **do** ▷ insert $\boldsymbol{x}$ at layer l
5 $\boldsymbol{C(x)} \leftarrow$ top-C candidates; ▷ greedy search
6 $\boldsymbol{N(x)} \leftarrow \leq R$ neighbors from $\boldsymbol{C(x)}$; ▷ heuristic strategy
7 add $\boldsymbol{x}$ to $\boldsymbol{N(y)}$ for each $\boldsymbol{y} \in \boldsymbol{N(x)}$; ▷ reverse edge
8 $\boldsymbol{V} \leftarrow \boldsymbol{V} \cup \{\boldsymbol{x}\}$, $\boldsymbol{E} \leftarrow \boldsymbol{E} \cup \boldsymbol{N(x)}$
9 **return** $\boldsymbol{G} = (\boldsymbol{V}, \boldsymbol{E})$

uppercase letters (e.g., $\boldsymbol{S}$) denote sets, while unbolded uppercase and lowercase letters represent parameters.

Given a vector dataset $\boldsymbol{S}$, the HNSW index $\boldsymbol{G}$ is built by progressively inserting vectors into the current graph index, as outlined in Algorithm 1. Two hyper-parameters must be set before index construction: the maximum number of candidates (C) and the maximum number of neighbors (R)[3]. For each inserted vector $\boldsymbol{x}$, the maximum layer l_{max} is randomly selected following an exponentially decaying distribution (line 3). The vector is inserted into layers from l_{max} to 0, with the same procedure applied at each layer (lines 4-7). At layer l, $\boldsymbol{x}$ is treated as a query point, and a greedy search on the current graph at layer l identifies the top-C nearest vertices as candidates (line 5). During the search, a candidate set $\boldsymbol{C(x)}$ of size C is maintained, with vertices ordered in ascending distance to $\boldsymbol{x}$, and the maximum distance is denoted as T. The search iteratively visits a vertex's neighbors, calculates their distances to $\boldsymbol{x}$, and updates $\boldsymbol{C(x)}$ with neighbors that have smaller distances ($< T$). To obtain the final neighbors, a heuristic edge selection strategy prevents clustering among neighbors (line 6). For example, for candidate $\boldsymbol{v}$ and any candidate $\boldsymbol{u}$ where $\delta(\boldsymbol{u}, \boldsymbol{x}) < \delta(\boldsymbol{v}, \boldsymbol{x})$, if $\delta(\boldsymbol{u}, \boldsymbol{v}) < \delta(\boldsymbol{v}, \boldsymbol{x})$, $\boldsymbol{v}$ is excluded from $\boldsymbol{N(x)}$. After determining $\boldsymbol{x}$'s final neighbors, $\boldsymbol{x}$ is added to the neighbor list $\boldsymbol{N(y)}$ for each $\boldsymbol{y} \in \boldsymbol{N(x)}$ (line 7). If $\boldsymbol{N(y)}$ exceeds R, the heuristic edge selection strategy prunes excess neighbors, ensuring that the neighbor count remains under R.

The construction process of HNSW involves two main steps: *Candidate Acquisition* (CA) and *Neighbor Selection* (NS). The CA step identifies the top-C nearest vertices for an inserted vector using a greedy search strategy. This involves visiting a vertex's neighbor list and computing distances to the inserted vector, requiring random memory access to fetch a neighbor's vector data, stored separately from neighbor IDs due to the large vector size. In the NS step, the final R neighbors are selected from the candidate set using a heuristic strategy. Here, distances among candidates are computed, also requiring random access to their vector data. In both CA and NS, distance calculations are used solely for comparison. In CA, distances are compared to the largest distance in the candidate set to determine if a visiting vertex can replace the farthest candidate. In NS, distances among candidates and between candidates and the inserted vector are compared to decide the inclusion of a candidate in the final neighbor set. HNSW currently performs all distance computations on full-precision vectors. Additionally, to utilize SIMD acceleration, a distance computation requires hundreds of read operations to load vector segments into a 128-bit register due to a large vector size, often spanning thousands of bytes.

[3]In the original paper and many popular repositories [2, 4, 78], C is called $efConstruction$, and R in the base layer is twice that in higher layers [78].

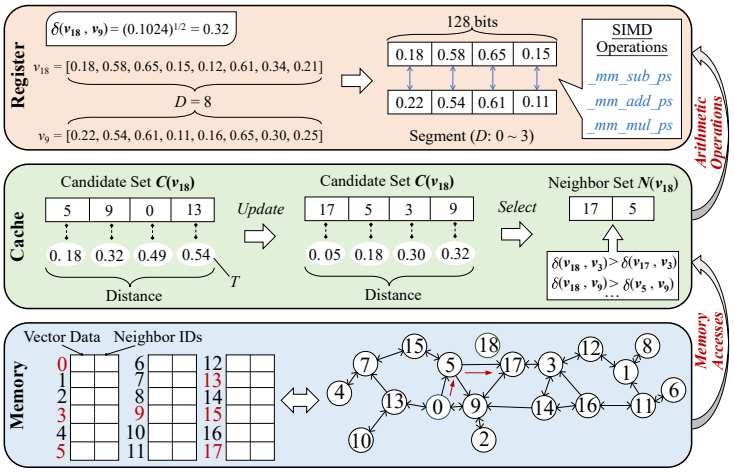


Fig. 2. Illustrating memory accesses and arithmetic operations during the HNSW construction (base layer).

Example 1. In Figure 2, we illustrate the index construction process of HNSW at the base layer, exemplified by inserting v_{18} with $C = 4$ and $R = 2$. In the CA step, v_{18} is treated as a query point, and a greedy search from v_0 is initiated. After examining v_0's neighbors, the candidate set $C(v_{18})$ is $\{v_5, v_9, v_0, v_{13}\}$, ordered by proximity to v_{18}. The current distance threshold T is $\delta(v_{13}, v_{18}) = 0.54$. The nearest vertex, v_5, is then visited, prompting an exploration of its neighbors to update $C(v_{18})$. Upon calculating $\delta(v_{15}, v_{18})$ and comparing it with T, v_{13} is replaced by v_{15} as $\delta(v_{15}, v_{18}) < T$, and T is updated to $\delta(v_0, v_{18}) = 0.49$. This process continues, resulting in the replacement of v_0 and v_{15} with v_{17} and v_3, respectively. At the end of CA, $C(v_{18})$ comprises $\{v_{17}, v_5, v_3, v_9\}$. The vertices v_0, v_3, v_5, v_9, v_{13}, v_{15}, and v_{17} are traversed in this phase. As shown in the graph index's memory layout (lower left), these vertices (highlighted in red) exhibit a random distribution, meaning numerous random memory accesses. In the NS step, v_{17} is initially chosen, leading to the removal of v_3 and v_9 from $C(v_{18})$ based on the conditions $\delta(v_{17}, v_3) < \delta(v_3, v_{18})$ and $\delta(v_{17}, v_9) < \delta(v_9, v_{18})$. The remaining vertex, v_5, is added to the neighbor list, which reaches the limit $R = 2$, yielding $\{v_{17}, v_5\}$. v_{18} is then appended to the neighbor lists of v_{17} and v_5, while ensuring that their neighbor counts remain below R through the heuristic strategy. In practice, $C(v_{18})$ can include thousands of candidates, making full vector caching impractical and necessitating many random memory accesses for distance computations during the NS stage. To leverage SIMD instructions, a vector is divided into multiple segments, each comprising four dimensions suitable for loading into a 128-bit register. While SIMD enhances processing speed, the necessity of numerous register load operations to sequentially fetch each vector segment leads to reduced SIMD utilization efficiency.

Our analysis reveals that the primary inefficiency in HNSW index construction lies in the current distance computation process, consuming over 90% of indexing time (Figure 1). This process suffers from numerous random memory accesses and suboptimal SIMD utilization, misaligning with modern CPU architecture. Thus, optimizing distance computation to better exploit modern CPU capabilities is essential for accelerating HNSW construction.

3 Methodology

In this section, we theoretically analyze how distance comparisons in HNSW construction can be effectively executed using compact vector codes. Based on this, we integrate three common vector compression methods—Product Quantization (PQ) [67], Scalar Quantization (SQ) [29], and Principal Component Analysis (PCA) [69]—into HNSW. While many variants of these methods

exist [51, 52, 57, 59, 99, 105, 109, 122], we exclude them due to their complexity compared to the original methods, which complicates deployment in HNSW. We focus on these three methods for their lightweight nature and alignment with our goal of accelerating index construction. Drawing from the integration of these methods in HNSW construction, we design a novel compact coding strategy and optimize the memory layout for HNSW construction, enhancing memory access efficiency and SIMD operations.

3.1 Theoretical Analysis

Recall that distance computation during HNSW construction is a major bottleneck, with one distance value primarily compared against another. We denote distance comparisons for updating the candidate set as DC1, and for selecting final neighbors as DC2. Here, we demonstrate that DC1 and DC2 can be unified into a generalized framework and analyze how these comparisons can be effectively performed using compact vector codes.

Lemma 1. *Given any three vertices $\boldsymbol{u}, \boldsymbol{v}$, and $\boldsymbol{w}$ in D-dimensional Euclidean space $\mathbb{R}^D$, the comparison between $\delta(\boldsymbol{u}, \boldsymbol{v})$ and $\delta(\boldsymbol{u}, \boldsymbol{w})$ is:*
- *$\delta(\boldsymbol{u}, \boldsymbol{v}) < \delta(\boldsymbol{u}, \boldsymbol{w})$ if and only if $\boldsymbol{e} \cdot \boldsymbol{u} - b < 0$;*
- *$\delta(\boldsymbol{u}, \boldsymbol{v}) > \delta(\boldsymbol{u}, \boldsymbol{w})$ if and only if $\boldsymbol{e} \cdot \boldsymbol{u} - b > 0$;*
- *$\delta(\boldsymbol{u}, \boldsymbol{v}) = \delta(\boldsymbol{u}, \boldsymbol{w})$ if and only if $\boldsymbol{e} \cdot \boldsymbol{u} - b = 0$;*

where $\boldsymbol{e} \cdot \boldsymbol{u} - b = 0$ defines the perpendicular bisector hyperplane between $\boldsymbol{v}$ and $\boldsymbol{w}$, with $\boldsymbol{e} = \boldsymbol{w} - \boldsymbol{v}$ and $b = \frac{\|\boldsymbol{w}\|^2 - \|\boldsymbol{v}\|^2}{2}$.

Proof. To show $\delta(\boldsymbol{u}, \boldsymbol{v}) = \delta(\boldsymbol{u}, \boldsymbol{w})$, we square both sides to eliminate square roots, giving $\|\boldsymbol{u} - \boldsymbol{v}\|^2 = \|\boldsymbol{u} - \boldsymbol{w}\|^2$. Expanding and simplifying, we get $2\boldsymbol{u} \cdot (\boldsymbol{w} - \boldsymbol{v}) = \|\boldsymbol{w}\|^2 - \|\boldsymbol{v}\|^2$. Defining $\boldsymbol{e} = \boldsymbol{w} - \boldsymbol{v}$ and $b = \frac{\|\boldsymbol{w}\|^2 - \|\boldsymbol{v}\|^2}{2}$, this simplifies to $\boldsymbol{e} \cdot \boldsymbol{u} = b$, the equation of a hyperplane in $\mathbb{R}^D$. For $\boldsymbol{u}$ to lie on this hyperplane, it must satisfy $\boldsymbol{e} \cdot \boldsymbol{u} - b = 0$. Similarly, for $\delta(\boldsymbol{u}, \boldsymbol{v}) > \delta(\boldsymbol{u}, \boldsymbol{w})$, squaring both sides gives $\|\boldsymbol{u} - \boldsymbol{v}\|^2 > \|\boldsymbol{u} - \boldsymbol{w}\|^2$, leading to $2\boldsymbol{u} \cdot (\boldsymbol{w} - \boldsymbol{v}) > \|\boldsymbol{w}\|^2 - \|\boldsymbol{v}\|^2$, and thus $\boldsymbol{e} \cdot \boldsymbol{u} - b > 0$. For $\delta(\boldsymbol{u}, \boldsymbol{v}) < \delta(\boldsymbol{u}, \boldsymbol{w})$, a similar process gives $\boldsymbol{e} \cdot \boldsymbol{u} - b < 0$. □

In DC1, with $\boldsymbol{u}$ as the newly inserted vector and $\boldsymbol{v}$ as the farthest vertex from $\boldsymbol{u}$ in the candidate set $\boldsymbol{C(u)}$, we update $\boldsymbol{C(u)}$ by including $\boldsymbol{w}$ if $\boldsymbol{e} \cdot \boldsymbol{u} - b > 0$. In DC2, with $\boldsymbol{v}$ as the newly inserted vector and $\boldsymbol{w}$ as a selected neighbor in $\boldsymbol{N(v)}$, we exclude a candidate $\boldsymbol{u}$ from consideration as a neighbor if $\boldsymbol{e} \cdot \boldsymbol{u} - b > 0$. Using $\boldsymbol{e} \cdot \boldsymbol{u} - b > 0$ as a criterion, both DC1 and DC2 can be correctly executed.

Theorem 1 ([29]). *For three vertices $\boldsymbol{u}, \boldsymbol{v}$, and $\boldsymbol{w}$ in D-dimensional Euclidean space $\mathbb{R}^D$, with compact vector codes $\boldsymbol{u}'$, $\boldsymbol{v}'$, and $\boldsymbol{w}'$, the condition $\delta(\boldsymbol{u}, \boldsymbol{v}) > \delta(\boldsymbol{u}, \boldsymbol{w})$ is equivalent to $\delta(\boldsymbol{u}', \boldsymbol{v}') > \delta(\boldsymbol{u}', \boldsymbol{w}')$ when $|\boldsymbol{e} \cdot \boldsymbol{u} - b| \geq |E|$. The quantity E is defined as*

$$\begin{aligned} E =& (E_{\boldsymbol{w}} - E_{\boldsymbol{v}}) \cdot \boldsymbol{u} + (\boldsymbol{w} - \boldsymbol{v}) \cdot E_{\boldsymbol{u}} + E_{\boldsymbol{v}} \cdot E_{\boldsymbol{u}} - E_{\boldsymbol{w}} \cdot E_{\boldsymbol{u}} \\ &+ \frac{1}{2}||E_{\boldsymbol{w}}||^2 - \frac{1}{2}||E_{\boldsymbol{v}}||^2 + \boldsymbol{v} \cdot E_{\boldsymbol{v}} - \boldsymbol{w} \cdot E_{\boldsymbol{w}} \end{aligned} \tag{1}$$

where $E_{\boldsymbol{u}}$, $E_{\boldsymbol{v}}$, and $E_{\boldsymbol{w}}$ are the error vectors corresponding to $\boldsymbol{u}$, $\boldsymbol{v}$, and $\boldsymbol{w}$, respectively.

Proof. Based on Lemma 1, the condition $\delta(\boldsymbol{u}', \boldsymbol{v}') > \delta(\boldsymbol{u}', \boldsymbol{w}')$ is equivalent to $\boldsymbol{e}' \cdot \boldsymbol{u}' - b' > 0$, where $\boldsymbol{e}' = \boldsymbol{w}' - \boldsymbol{v}'$ and $b' = \frac{\|\boldsymbol{w}'\|^2 - \|\boldsymbol{v}'\|^2}{2}$. To demonstrate that $\delta(\boldsymbol{u}, \boldsymbol{v}) > \delta(\boldsymbol{u}, \boldsymbol{w})$ is equivalent to $\delta(\boldsymbol{u}', \boldsymbol{v}') > \delta(\boldsymbol{u}', \boldsymbol{w}')$, it suffices to prove that $\boldsymbol{e} \cdot \boldsymbol{u} - b$ and $\boldsymbol{e}' \cdot \boldsymbol{u}' - b'$ share the same sign. Substituting $\boldsymbol{e}'$ and b' into $\boldsymbol{e}' \cdot \boldsymbol{u}' - b'$, we obtain:

$$\boldsymbol{e}' \cdot \boldsymbol{u}' - b' = (\boldsymbol{w}' - \boldsymbol{v}') \cdot \boldsymbol{u}' - \frac{\|\boldsymbol{w}'\|^2 - \|\boldsymbol{v}'\|^2}{2} \tag{2}$$

For vector $\boldsymbol{u}$, let $E_{\boldsymbol{u}}$ denote the compression error of $\boldsymbol{u}$, defined as

$$E_{\boldsymbol{u}} = \boldsymbol{u} - \boldsymbol{u}' \tag{3}$$

Substituting this into the previous equation, we derive:

$$\begin{aligned} \boldsymbol{e}' \cdot \boldsymbol{u}' - b' =& ((\boldsymbol{w} - E_{\boldsymbol{w}}) - (\boldsymbol{v} - E_{\boldsymbol{v}})) \cdot (\boldsymbol{u} - E_{\boldsymbol{u}}) \\ & - \frac{\|\boldsymbol{w} - E_{\boldsymbol{w}}\|^2 - \|\boldsymbol{v} - E_{\boldsymbol{v}}\|^2}{2} \end{aligned} \tag{4}$$

Expanding and simplifying, we group terms involving compression errors:

$$\begin{aligned} E =& (E_{\boldsymbol{w}} - E_{\boldsymbol{v}}) \cdot \boldsymbol{u} + (\boldsymbol{w} - \boldsymbol{v}) \cdot E_{\boldsymbol{u}} + E_{\boldsymbol{v}} \cdot E_{\boldsymbol{u}} - E_{\boldsymbol{w}} \cdot E_{\boldsymbol{u}} \\ & + \frac{1}{2}||E_{\boldsymbol{w}}||^2 - \frac{1}{2}||E_{\boldsymbol{v}}||^2 + \boldsymbol{v} \cdot E_{\boldsymbol{v}} - \boldsymbol{w} \cdot E_{\boldsymbol{w}} \end{aligned} \tag{5}$$

Thus, the equation simplifies to:

$$\begin{aligned} \boldsymbol{e}' \cdot \boldsymbol{u}' - b' &= (\boldsymbol{w} - \boldsymbol{v}) \cdot \boldsymbol{u} - \frac{1}{2}(||\boldsymbol{w}||^2 - ||\boldsymbol{v}||^2) - E \\ &= \boldsymbol{e} \cdot \boldsymbol{u} - b - E \end{aligned} \tag{6}$$

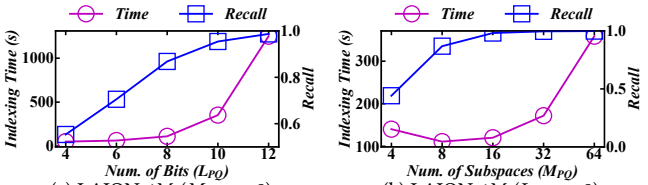
Therefore, when $|\boldsymbol{e} \cdot \boldsymbol{u} - b| \geq |E|$, the sign of $\boldsymbol{e}' \cdot \boldsymbol{u}' - b'$ matches that of $\boldsymbol{e} \cdot \boldsymbol{u} - b$. Specifically, $\boldsymbol{e}' \cdot \boldsymbol{u}' - b' > 0$ if and only if $\boldsymbol{e} \cdot \boldsymbol{u} - b > 0$, and similarly, $\boldsymbol{e}' \cdot \boldsymbol{u}' - b' < 0$ if and only if $\boldsymbol{e} \cdot \boldsymbol{u} - b < 0$. □

Theorem 1 elucidates how distance comparison is preserved despite compression error. By adjusting the parameters of compact coding (thus affecting E), the condition $|\boldsymbol{e} \cdot \boldsymbol{u} - b| \geq |E|$ can always be satisfied. Consequently, an appropriate compression error maintains accurate distance comparison while reducing vector data size, enhancing memory access and SIMD operations. Hence, integrating vector compression techniques into the HNSW construction process is a promising avenue for accelerating index construction. Given a vector $\boldsymbol{u}$ and its compact vector code $\boldsymbol{u}'$, the error vector is computed as $E_{\boldsymbol{u}} = \boldsymbol{u} - \boldsymbol{u}'$. Here, $\boldsymbol{u}'$ refers to the vector derived from the compact vector code, not the vector code itself; this derived vector retains the same dimensionality as $\boldsymbol{u}$. For different compact coding methods, the derived vectors have different interpretations, and thus $E_{\boldsymbol{u}}$ is computed in distinct ways (see Section 3.2).

In a compact coding method, adjustable parameters control the compression error. Specifically, a random sample of vectors (e.g., 10,000) is drawn from the dataset, and each vector's top-100 nearest neighbors are determined, forming a triple $(\boldsymbol{u}, \boldsymbol{v}, \boldsymbol{w})$ from the vector and its two nearest neighbors. A batch of such triples is then constructed, followed by the generation of compact vector codes $\boldsymbol{u}'$, $\boldsymbol{v}'$, and $\boldsymbol{w}'$ for specific parameters. Next, the quantities $|\boldsymbol{e} \cdot \boldsymbol{u} - b|$ and $|E|$ are computed for each triple. By tuning the parameters, one can maximize the proportion of triples that satisfy $|\boldsymbol{e} \cdot \boldsymbol{u} - b| \geq |E|$ while minimizing the vector size. Finally, the chosen parameters are evaluated by integrating the compact coding method into HNSW.

3.2 Baseline Solutions

3.2.1 ***Deploying PQ in HNSW.*** Product Quantization (PQ) [67] splits high-dimensional vectors into subvectors that are independently compressed. For each subspace, a codebook of centroids is generated, and during encoding, the ID of the nearest centroid is selected to create a compact representation. The trade-off between distance comparison accuracy and compression error is managed by adjusting the code length (L_{PQ}) in each subspace and the number of subspaces (M_{PQ}). For example, with $M_{PQ} = 8$ and $L_{PQ} = 4$, a vector is represented by 8 codewords, each storing a 4-bit centroid ID in the subspace. For a vector $\boldsymbol{u}$ and its compact code $\boldsymbol{u}'$, the derived vector is



(a) LAION-1M (M_{PQ} = 8) (b) LAION-1M (L_{PQ} = 8)

Fig. 3. Effect of parameters on HNSW-PQ.

formed by concatenating $\boldsymbol{u}$'s nearest centroid vectors (identified by $\boldsymbol{u'}$) from all subspaces. The error vector $E_{\boldsymbol{u}}$ is then computed as the difference between $\boldsymbol{u}$ and this derived vector.

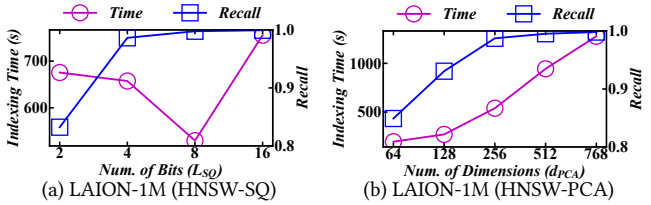
To incorporate PQ into HNSW, we preprocess high-dimensional vectors to create codebooks. Each inserted vector is encoded using these codebooks, and a distance table is computed between the vector and subspace centroids (Asymmetric Distance Computation, ADC [67]). In the Candidate Acquisition (CA) stage, distances to visited vertices are efficiently computed by scanning this table. In the Neighbor Selection (NS) stage, the table cannot compute candidate distances. Thus, centroids are located based on compact PQ codes, and inter-centroid distances represent candidate distances (Symmetric Distance Computation, SDC [67]). Theorem 1 suggests that optimal values of L_{PQ} and M_{PQ} balance distance comparison accuracy with construction efficiency. We denote the HNSW constructed via this pipeline as HNSW-PQ, which replaces original vectors with PQ codes and employs ADC and SDC for efficient distance computation during index construction.

We evaluate HNSW-PQ under different L_{PQ} and M_{PQ} configurations, as shown in Figure 3. The results indicate that both parameters significantly influence indexing time and search performance. As L_{PQ} grows, more clusters are formed in each subspace, leading to longer codebook generation and vector encoding times, thus increasing indexing time. Notably, indexing time initially decreases before rising with M_{PQ}. This occurs because a small M_{PQ} causes high subspace dimensionality, increasing codebook generation time. Conversely, a larger M_{PQ} also raises indexing time due to more subspaces. Regarding index quality, both higher L_{PQ} and M_{PQ} improve recall, aligning with the theoretical analysis that lower compression errors yield more accurate distance comparisons.

3.2.2 ***Deploying SQ in HNSW.*** Scalar Quantization (SQ) [29, 36] compresses each dimension of vectors independently. It divides data ranges into intervals, assigning each interval an integer value. Floating-point values within an interval are approximated by that integer value, thus reducing vector size by lowering the required bits (L_{SQ}). For example, with L_{SQ} = 8, each dimension is represented by an integer (0~255). Before distance computation, these integers are converted back into floating-point numbers to form the decoded vector, which is lossy. The error vector $E_{\boldsymbol{u}}$ is obtained by subtracting the decoded vector from $\boldsymbol{u}$.

To accelerate HNSW construction via SQ, we preprocess high-dimensional vectors to identify the maximum and minimum values per dimension. We calculate the interval for each dimension by subtracting the minimum from the maximum. During index construction, each inserted vector is encoded using these precomputed intervals, producing compact SQ codes for distance computations. As per Theorem 1, the optimal L_{SQ} balances distance comparison accuracy with construction efficiency. The resulting HNSW, termed HNSW-SQ, replaces original vector data with compact SQ codes, enabling efficient distance calculations.

In Figure 4(a), we assess HNSW-SQ across varying L_{SQ} values. The results show that indexing time first decreases and then increases as L_{SQ} grows. This trend arises from the lack of specialized data types for 2 and 4 bits, necessitating the use of *uint8* type, which reduces operational efficiency. In contrast, 8 bits, while larger, aligns well with *uint8*, achieving an optimal balance between compression error and operational efficiency, thereby reducing indexing time. While 16 bits can be



(a) LAION-1M (HNSW-SQ) (b) LAION-1M (HNSW-PCA)

Fig. 4. Effect of parameters on HNSW-SQ and HNSW-PCA.

represented with *uint16*, the larger vector size increases indexing time. Notably, search accuracy consistently improves with higher L_{SQ}, supporting the theoretical analysis.

3.2.3 ***Deploying PCA in HNSW.*** Principal Component Analysis (PCA) [69] reduces dimensionality by projecting high-dimensional vectors into a lower-dimensional space. It applies an orthogonal transformation matrix to a vector $\boldsymbol{u}$, preserving its norm while prioritizing significant components in lower dimensions. The compact vector code $\boldsymbol{u'}$ is formed by retaining the first d_{PCA} dimensions, substantially reducing storage requirements. The error vector $E_{\boldsymbol{u}}$ is calculated by subtracting the dimension-reduced vector from the transformed vector of $\boldsymbol{u}$, with the dimension-reduced vector zero-padded to match dimensionality.

To expedite HNSW construction via PCA, we preprocess high-dimensional vectors by deriving an orthogonal projection matrix through eigenvalue decomposition of the covariance matrix. Each inserted vector is then transformed into a low-dimensional space using this matrix, with principal components serving as compact PCA codes. Distance computations are performed directly on these compact representations. Theorem 1 indicates that an optimal d_{PCA} balances distance comparison accuracy and construction efficiency. The resulting HNSW, designated as HNSW-PCA, substitutes the original vectors with compact PCA representations, thereby facilitating more efficient distance computations.

Figure 4(b) shows the evaluation of HNSW-PCA across various d_{PCA} configurations. We find that indexing time increases with higher d_{PCA}, while search accuracy improves, aligning with the theoretical analysis. Optimal trade-off between indexing time and search performance occurs with d_{PCA} ranging from 256 to 512. Notably, d_{PCA} reaches 420 when 90% cumulative variance is achieved on the LAION dataset. In the experiments, d_{PCA} will be set to ensure at least 90% cumulative variance.

3.2.4 ***Lessons Learned.*** Theoretical and empirical analyses reveal that index construction can be accelerated using compact coding techniques, and construction efficiency and index quality can be well balanced by adjusting the compression error. An appropriate compression error reduces vector data size, improving memory access and SIMD operation efficiency while preserving accurate distance comparisons. Following this insight, optimized variants [36, 51, 52, 57, 59, 99, 105, 109, 122] of PQ, SQ, and PCA may be integrated into HNSW to further speed up index construction. Note that these variants must avoid excessive processing overhead to maintain the balance between construction efficiency and index quality. Nonetheless, the core mechanism of these methods—vector size reduction—does not align well with HNSW's construction characteristics on modern CPUs. Regardless of whether HNSW-PQ, HNSW-SQ, or HNSW-PCA is used, the random memory access pattern during index construction remains unchanged. In terms of arithmetic operations, HNSW-PQ's distance table lookup is unable to fully utilize SIMD operations, as each computation is executed individually [51]. While HNSW-SQ and HNSW-PCA reduce register loads by computing directly on compressed vectors, they still process distance computations sequentially, underutilizing SIMD's advantages [30]. We highlight that current variants of PQ, SQ, and PCA do not fundamentally resolve these limitations.

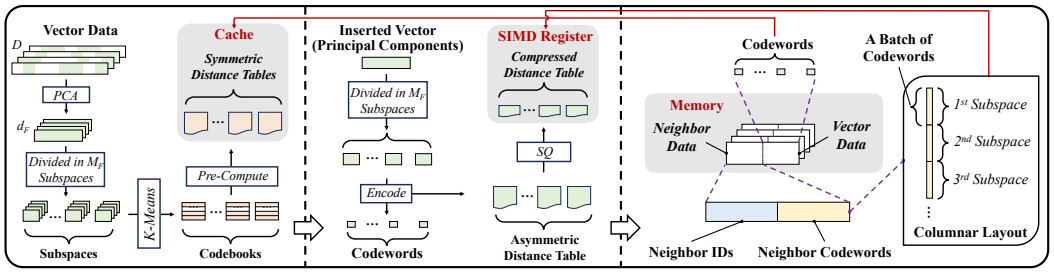


Fig. 5. Illustrating the coding pipeline and data layout of Flash.

3.3 The Flash Method

In Section 3.2, we demonstrate that incorporating existing compact coding methods into HNSW construction can enhance index construction speed. However, these methods either offer limited gains in indexing efficiency or degrade search performance. This is mainly because they are unable to effectively reduce random memory accesses and fully exploit the advantages of SIMD instructions. To overcome these limitations, we propose a specific compact coding strategy alongside memory layout optimization for the construction process of HNSW, named Flash, which minimizes random memory accesses while maximizing SIMD utilization.

3.3.1 ***Design Overview.*** Noting that HNSW-PQ strikes an optimal balance between construction efficiency and search performance, with PQ offering flexibility through two adjustable parameters that affect the compression error. We leverage the core principles of PQ to devise Flash, tailored to the HNSW construction process on modern CPUs. Figure 5 provides an overview of the coding pipeline and data layout used by Flash.

Given that a CPU core has multiple SIMD registers, Flash partitions the high-dimensional space into subspaces, enabling parallel processing of subspaces in these registers. To accommodate the distinct distance computations in Candidate Acquisition (CA) and Neighbor Selection (NS) stages, it introduces Asymmetric Distance Tables (ADT) for the CA stage, residing in SIMD registers, and Symmetric Distance Tables (SDT) for the NS stage, located in cache. It optimizes the bit counts allocated for each codeword (i.e., encoded vector) and applies SQ to compress the precomputed ADT to fit within a SIMD register, thereby reducing register loads. Since high-dimensional vectors often exhibit uneven variances across dimensions, directly encoding original subspaces may result in suboptimal bit utilization. To address this, Flash utilizes PCA to extract the principal components of vectors, generating subspaces based on these components, enhancing bit utilization within the SIMD register's constraints. When organizing neighbor lists, neighbor IDs are grouped with corresponding codewords to minimize random memory accesses. Rather than sequentially storing all codewords for a neighbor, Flash gathers the codewords of a batch of neighbors within a specific subspace. The batch size matches the number of centroids per subspace. This alignment allows a register load to fetch an entire batch, enabling partial distances to be obtained in parallel using SIMD shuffle operations since the ADT for a subspace is fully contained within a SIMD register.

Flash enhances HNSW construction by optimizing memory layout and SIMD operations. Two hyperparameters—the number of subspaces (M_F) and the dimension of the principal components (d_F)—can be adjusted to balance construction efficiency and search performance (Theorem 1). These adjustments do not affect Flash's memory access pattern or SIMD optimizations. We highlight that existing SIMD optimizations for PQ, as discussed in [30, 31], target linear scans or inverted file (IVF) structures, where vector batches are naturally grouped, rather than graph index settings [51].

3.3.2 ***Extracting Principal Components.*** To align with SIMD register constraints, each subspace has a limited bit representation (e.g., 4 bits). Encoding on principal components optimally utilizes these bits, reducing quantization error. Thus, `Flash` employs PCA to project high-dimensional vectors by rearranging dimensions in descending order of variance. This approach preserves essential low-dimensional principal components by selecting the first d_F dimensions, encapsulating the most significant features. Given a dataset $\boldsymbol{S}$ of n vectors, each with D dimensions, the mean vector is $\bar{\boldsymbol{u}} = \frac{1}{n}\sum_{i=1}^{n} \boldsymbol{u}_i$. Data is centered by subtracting $\bar{\boldsymbol{u}}$ from each vector, producing the centered matrix $\dot{\boldsymbol{S}} = \boldsymbol{S} - \mathbf{1} \cdot \bar{\boldsymbol{u}}^\top$, where $\mathbf{1}$ is a unit vector. The covariance matrix $\Sigma = \frac{1}{n}(\dot{\boldsymbol{S}}^\top \dot{\boldsymbol{S}})$ is then computed, and its eigenvalues $\lambda_1, \lambda_2, \ldots, \lambda_D$ and eigenvectors $\boldsymbol{a}_1, \boldsymbol{a}_2, \ldots, \boldsymbol{a}_D$ are extracted. For a given variance fraction α, the function $f(d) = \frac{\sum_{i=1}^{d} \lambda_i}{\sum_{i=1}^{D} \lambda_i}$ identifies the smallest d_F such that $f(d_F) \geq \alpha$, defining the principal component space of dimension d_F. The principal components of each vector are established based on the basis $\boldsymbol{A}_{1:d_F} = (\boldsymbol{a}_1\ \boldsymbol{a}_2\ \cdots\ \boldsymbol{a}_{d_F})$, yielding the reduced-dimensional data $\widetilde{\boldsymbol{S}}$:

$$\widetilde{\boldsymbol{S}} = \{\tilde{\boldsymbol{u}}_i \mid \tilde{\boldsymbol{u}}_i = \mathrm{A}_{1:d_F}^\top \boldsymbol{u}_i, \text{ for } i = 1, \ldots, n\} \tag{7}$$

3.3.3 ***Subspace Division and Distance Table Compression.*** `Flash` decomposes each vector's principal components $\tilde{\boldsymbol{u}}$ into M_F disjoint subvectors, denoted as $\tilde{\boldsymbol{u}} = [\tilde{\boldsymbol{u}}_1, \tilde{\boldsymbol{u}}_2, \ldots, \tilde{\boldsymbol{u}}_{M_F}]$, where each $\tilde{\boldsymbol{u}}_i$ resides in a specific subspace of the principal component domain. Similar to PQ, a codebook $\boldsymbol{C}_i = \{\boldsymbol{c}_{i1}, \boldsymbol{c}_{i2}, \ldots, \boldsymbol{c}_{iK}\}$ is created for each subspace, with $\boldsymbol{c}_{ij}$ denoting a codeword (i.e., a centroid ID) in the i-th subspace, and K representing the number of centroids. The i-th subvector $\tilde{\boldsymbol{u}}_i$ is quantized by identifying the closest centroid in the codebook $\boldsymbol{C}_i$ of the i-th subspace:

$$\psi(\tilde{\boldsymbol{u}}_i) = \arg\min_{\boldsymbol{c}_{ij} \in \boldsymbol{C}_i} \delta(\tilde{\boldsymbol{u}}_i, \boldsymbol{c}_{ij}) \tag{8}$$

where $\psi(\tilde{\boldsymbol{u}}_i)$ is the nearest centroid's ID serving as the codeword of the subvector. Consequently, the complete principal components $\tilde{\boldsymbol{u}}$ are represented as a sequence of codewords, one for each subvector. The codeword length L_F relates to the number of centroids K via $L_F = \lceil log_2 K \rceil$ in a subspace. `Flash` samples a subset from the complete vector set to efficiently construct codebooks, following PQ and its variants [52, 67].

For each inserted vector $\boldsymbol{u}$, asymmetric distance tables (ADT) and codewords are simultaneously generated across M_F subspaces using precomputed codebooks, leveraging shared distance computations between each subvector $\tilde{\boldsymbol{u}}_i$ and the centroids in $\boldsymbol{C}_i$. This approach avoids duplicate calculations, enhancing efficiency. In the Candidate Acquisition (CA) stage, the ADT enables fast summation of partial distances to calculate distances between $\boldsymbol{u}$ and visited vertices. To adapt ADT for SIMD registers, `Flash` employs SQ to map each distance $dist$ in the table to discrete levels:

$$\eta(dist) = \left\lfloor \left(\frac{dist - dist_{\min}}{\Delta} \right) \cdot (2^H - 1) \right\rfloor \tag{9}$$

where $\eta(dist)$ is the quantized distance, $dist_{\min}$ is the minimum distance, Δ is the quantization step size ($\Delta = dist_{max} - dist_{min}$), and H is the bit number for a quantized distance value. With $K = 16$ and $H = 8$, each ADT is 128 bits in size. These ADTs are stored as *one-dimensional arrays* that can reside in registers during the insertion process, and are freed once the insertion is completed.

In the Neighbor Selection (NS) stage, distance computations among candidates preclude the use of ADT. To address this, `Flash` precomputes a symmetric distance table (SDT) that stores distances between centroids in each subspace. With M_F subspaces and K centroids, M_F SDTs are built, each holding K^2 distance values in a *two-dimensional array*. These values are quantized similarly to ADT for optimal caching. `Flash` obtains distances between candidates from SDTs based on their codewords. Note that SDTs are shared by all inserted vectors during index construction, eliminating numerous random memory accesses. To compare distance values from SDTs and ADTs for neighbor

selection, Flash uses the same quantization step size Δ and target bit number H for both tables. It calculates the maximum and minimum distances ($dist^i_{max}$, $dist^i_{min}$) within each subspace, summing each $dist^i_{max}$ to obtain $dist_{max}$ and taking the lowest among $dist^i_{min}$ as $dist_{min}$.

3.3.4 ***Access-Aware Memory Layout.*** In the graph index, all vertices share a uniform neighbor list structure and size. A *contiguous memory block* is allocated for each vertex's vector and neighbor data, accessed via an offset determined by the vertex ID. For an inserted vector $\boldsymbol{u}$, the CA stage updates candidates via greedy search, visiting a vertex's neighbors in the sub-graph index and computing their distances to $\boldsymbol{u}$. Current graph indexes suffer from numerous random accesses when fetching neighbor vectors, as neighbor vectors are too big to be stored alongside neighbor IDs. By attaching neighbor codewords to these IDs, Flash computes distances directly in register-resident ADTs, avoiding random memory accesses. For each vertex, Flash stores the neighbor IDs in one fixed memory block, followed by the neighbor codewords in another (see Figure 5, lower right). To efficiently use SIMD instructions, Flash gathers B neighbor codewords in a subspace, processing them concurrently to ensure efficient SIMD register loading and parallel distance computations for B neighbors.

3.3.5 ***SIMD Acceleration.*** SIMD instructions support vectorized execution, enabling simultaneous processing of multiple data points. This can accelerate distance calculations between the inserted vector $\boldsymbol{u}$ and a batch of neighbors. Flash uses SIMD to look up ADTs and sum partial distances across subspaces. An ADT resides in a SIMD register, storing distances between $\boldsymbol{u}$ and centroids in each subspace. Flash employs shuffle operations to extract partial distances using neighbors' codewords. In a single operation, codewords for B neighbors are loaded into a register, which then serves as indices to access the partial distances in the ADT. Specifically, the shuffle operation rearranges the data within the register according to the indices, extracting partial distances from the ADT without complex conditional branching or excessive memory accesses. Partial distances from two subspaces are summed using SIMD instructions, and this process is repeated across all subspaces to compute the final distances for the B neighbors. By using SIMD lookups for ADTs, Flash minimizes random memory accesses and exploits high-speed registers for parallel arithmetic, thereby improving computational efficiency.

3.3.6 ***Implementation on HNSW.*** We integrate Flash into the HNSW algorithm to accelerate indexing and search processes. It be- gins by preprocessing the dataset to extract principal components (Section 3.3.2), then sampling a subset, dividing it into subspaces, generating codebooks, and pre-computing centroid distances for the SDT. During index construction (lines 2-8 of Algorithm 1), each vector $\boldsymbol{u}$ has its principal components partitioned by the codebooks. Distances between each subvector $\boldsymbol{u}_i$ and codebook centroids $\boldsymbol{C}_i$ form the ADT, and the nearest centroid ID is selected as the codeword for $\boldsymbol{u}_i$. The ADT is quantized (Section 3.3.3) and held in a register until insertion completes. For line 5, SIMD acceleration computes distances between $\boldsymbol{u}$ and batches of neighbors based on ADT (Section 3.3.5). For lines 6–7, distances between candidates are approximated via SDT lookups, using each candidate's codewords as indices. We tune the number of subspaces (M_F) and the dimensionality of principal components (d_F) to manage compression error (Theorem 1) without affecting Flash's efficient memory access or SIMD operations. Other fixed parameters include bits per codeword (L_F) and batch size (B), set by the SIMD register size. The search procedure follows the CA paradigm, and we apply an additional reranking step based on original vectors to refine results.

Remarks. (1) Although many studies optimize the compression error of PQ, SQ, and PCA [29, 52, 57, 119], none are specifically tailored to meet the distinct requirements of graph indexing on modern CPU architectures. Integrating these compression methods into the graph construction

process often yields limited efficiency gains or even degrades search performance, as they neither address random memory access patterns nor leverage SIMD efficiently. In contrast, `Flash` is a specialized compact coding strategy designed for graph index construction and optimized for current CPU architectures. Notably, `Flash` incorporates the complementary principles of established vector compression methods (e.g., PQ, SQ, PCA) within its framework and offers the flexibility to integrate new optimizations of these compression methods. While several studies integrate vector compression into search processes [29, 66, 103, 116], search differs markedly from index construction, which involves more complex tasks such as the NS stage. Moreover, it is crucial that compression methods avoid excessive overhead, maintaining a balance between construction speed and index quality. Recent work [66, 103, 113, 116] shows that boosting search performance often increases indexing time. (2) In `Flash`, codeword and ADT generations share identical distance computations (Section 3.3.3), prompting an integrated implementation to eliminate redundancies. We utilize the Eigen library for matrix manipulations throughout data processing, including principal component extraction, codebook generation, and distance table creation. When the maximum number of neighbors R exceeds the batch size B, we split each vertex's neighbor data into multiple blocks, each matching the batch size. This structure enables the efficient distance computation within each block using SIMD instructions.

3.3.7 ***Cost Analysis.*** In the CA stage, the time complexity of HNSW is approximately $O(R{\cdot}log(n))$, where R is the maximum number of neighbors and $log(n)$ denotes the search path length [75, 104]. For each hop, vector data of the visiting vertices' neighbors must be accessed. Therefore, the number of memory accesses in the original HNSW algorithm (NMA_{orig}) can be expressed as:

$$NMA_{orig} \sim O(R \cdot log(n)) \tag{10}$$

In our optimized implementation, neighbors' codewords and their respective IDs are stored contiguously, eliminating the need for random memory accesses to fetch neighbor vectors. As a result, the number of memory accesses in our implementation (NMA_{ours}) is:

$$NMA_{ours} \sim O(log(n)) \tag{11}$$

For simplicity, this analysis excludes cache hits. Notably, the `Flash` code is significantly smaller than the original vector data, resulting in a higher cache hit ratio. In the NS stage, computing distances between candidates requires accessing their vector data. For the original HNSW algorithm, the number of memory accesses is $O(C)$, where C denotes the size of the candidate set. In contrast, `Flash` maintains the entire SDT in the L1 cache, avoiding fetching vector data from main memory during the NS stage.

For each distance computation using SIMD acceleration, the number of register loads in the original HNSW (NRL_{orig}) is:

$$NRL_{orig} = \frac{32 \cdot D}{U} \tag{12}$$

where D is the vector dimensionality (each vector has D floating-point values) and U is the bit-width of a register. This is because each vector segment must be individually loaded into an SIMD register. Furthermore, complex arithmetic operations, including subtraction, multiplication, and addition, are required for distance computation. In contrast, the number of register loads in our implementation (NRL_{ours}) is

$$NRL_{ours} = \frac{M_F \cdot H}{U} \tag{13}$$

where M_F is the number of subspaces and H represents the number of bits to encode a partial distance within each subspace. In `Flash`, the ADT is register-resident. Thus, it loads the codewords of B neighbors within a subspace into a register in a single operation. To compute distances for

Table 1. Statistics of experimental datasets.

Datasets	Data Volume	Size (GB)	Dim.	Query Volume
ARGILLA [16]	21,071,228	81	1,024	100,000
ANTON [15]	19,399,177	75	1,024	100,000
LAION [24]	100,000,000	293	768	100,000
IMAGENET [8]	13,158,856	38	768	100,000
COHERE [7]	10,124,929	30	768	100,000
DATACOMP [9]	12,800,000	37	768	100,000
BIGCODE [17]	10,404,628	30	768	100,000
SSNPP [6]	1,000,000,000	960	256	100,000

these neighbors, M_F batches of codewords are loaded, corresponding to M_F register loads. The parameter B is typically set to U/H to align with register capacity. Additionally, the arithmetic operations in `Flash` are reduced to simple addition. With $D = 768$, $U = 128$, $M_F = 16$, and $H = 8$, the number of register loads required for a distance computation in the original HNSW is 192, whereas in our optimized version, it is reduced to just 1.

4 Evaluation

We conduct a comprehensive experimental evaluation to answer the following research questions:
Q1: What impact do mainstream compact coding algorithms have on HNSW construction? (Sections 4.2 and 4.3)
Q2: How does `Flash` affect HNSW's construction efficiency, index size, and search performance? (Sections 4.2 and 4.3)
Q3: How does `Flash` scale across varying data volumes and segment counts (Section 4.4)?
Q4: How general is `Flash` with respect to various graph algorithms, HNSW optimizations, and SIMD instructions (Section 4.5)?
Q5: To what extent does `Flash` optimize memory accesses and arithmetic operations during index construction? (Section 4.6)
Q6: How do the parameters of `Flash` influence construction efficiency and index quality? (Section 4.7)

All source codes and datasets are publicly available at: https://github.com/ZJU-DAILY/HNSW-Flash.

4.1 Experimental Settings

4.1.1 ***Dataset.*** We use eight real-world vector datasets of diverse volumes and dimensions from deep embedding models, all of which are publicly accessible on Hugging Face and Big ANN Benchmarks websites [6, 20]. A detailed summary of these datasets is presented in Table 1. The query set is sampled from the dataset, with the corresponding ground truth generated through a linear scan.

4.1.2 ***Compared Methods.*** We employ an advanced HNSW implementation from the latest *hnswlib* repository [2] as our benchmark, recognized as a standard in the field [47, 70]. This implementation is widely referenced by industrial vector databases [19, 23, 37] and is a preferred choice for ANNS research [50, 62, 89]. To accelerate index construction, three established compact coding methods are integrated into the repository, forming three baseline solutions. Additionally, the proposed `Flash` method is also incorporated into the repository for fair comparisons. The generality of `Flash` is further evaluated on two recently optimized HNSW implementations (ADSampling [50] and VBase [121]) and two additional graph algorithms (NSG [49] and τ-MG [88]).

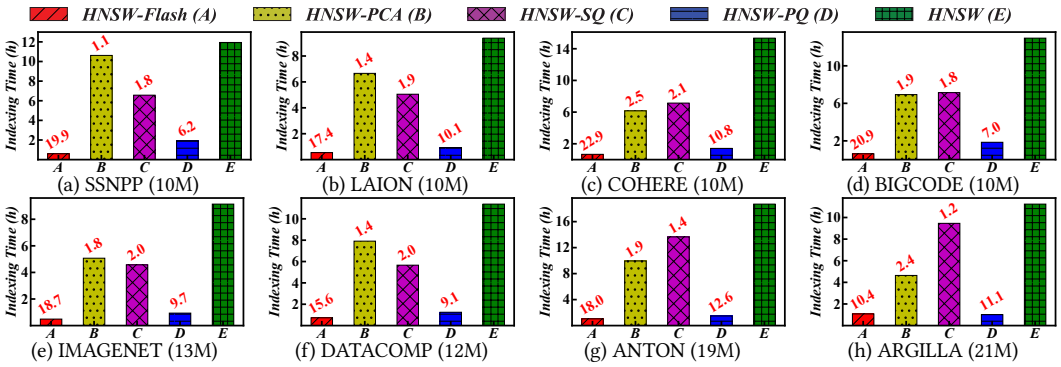


Fig. 6. Indexing times for all compared methods (red values at the top of each bar indicate speedup ratios).

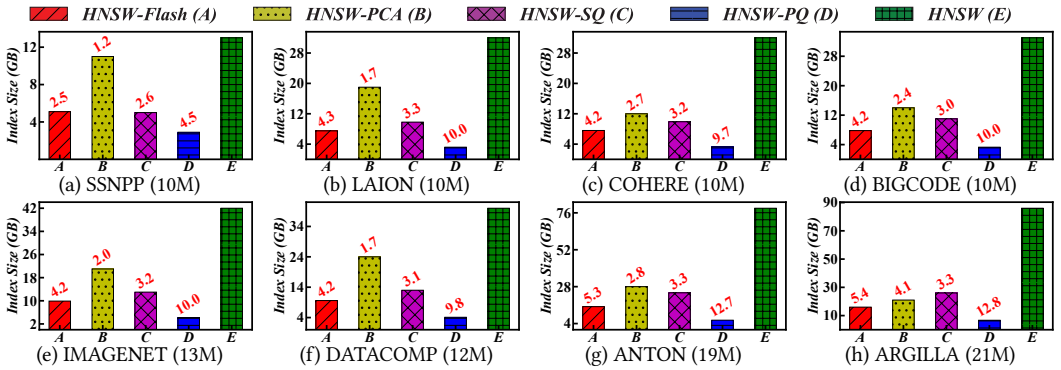


Fig. 7. Index sizes for all compared methods (red values at the top of each bar indicate compression ratios).

• **HNSW** [78]. HNSW is a prominent graph-based ANNS algorithm [50, 73, 104]. Its prevalence in industrial applications [108, 114, 121] and extensive academic research [71, 123, 128] underscore its significance (see Section 2.1). Given its representativeness, HNSW serves as a key graph index example in this research.

• **HNSW-PQ**. Product Quantization (PQ) is a recognized vector compression method in high-dimensional vector search [52, 57, 67]. We integrate PQ into the HNSW index construction process to accelerate graph indexing (see Section 3.2.1 for details).

• **HNSW-SQ**. Scalar Quantization (SQ) converts floating-point values to integers, reducing memory overhead and enhancing search performance [10, 29, 96]. We integrate it into HNSW construction according to Section 3.2.2, and implement an optimized version to avoid decoding overhead based on a recent technical report [10].

• **HNSW-PCA**. Principal Component Analysis (PCA) is a classical dimensionality reduction method that enhances calculation efficiency by reducing the need to scan all dimensions [50, 113]. We integrate PCA into the construction process following Section 3.2.3.

• **HNSW-Flash**. We incorporate the proposed compact coding method, `Flash`, into the HNSW construction process. This method thoughtfully considers the unique characteristics of HNSW construction, optimizing memory layout and arithmetic operations to minimize random memory accesses and enhance SIMD utilization.

4.1.3 ***Parameter Settings.*** For all methods compared, we set the maximum number of candidates (C) to 1024 and the maximum number of neighbors (R) to 32, following prior research

recommendations[75]. For compact coding parameters, we use established conventions to determine optimal settings. Specifically, the dimension of principal components (d_{PCA}) is chosen to ensure accumulated variance exceeds 0.9 in HNSW-PCA, as guided by our evaluations (see Section 3.2.3). For HNSW-SQ, we set the number of bits (L_{SQ}) to 8, which has been demonstrated as optimal in our experiments and existing literature [10, 96]. In HNSW-PQ, the number of subspaces (M_{PQ}) is identified through grid search, with bits per subspace (L_{PQ}) set to 8, consistent with common configurations in the literature [30, 52, 67, 118]. For HNSW-Flash, we maintain the same subspace count (M_F) as HNSW-PQ; and the dimension of principal components (d_F) is selected via grid search. The bits per subspace (L_F) are set to 4, while each distance value is allocated 8 bits ($H = 8$), enabling the ADT within a subspace to fit entirely within a SIMD register. Additionally, the batch size (B) for organizing the neighbor list is set to 16 to adapt to the number of distances within an ADT ($B = K = 2^{L_F}$) and optimize register loading, resulting in each neighbor list comprising two blocks for separate execution of distance computations ($R/B = 2$).

4.1.4 ***Performance Metrics.*** To evaluate index construction efficiency, we record indexing time, which includes data preprocessing times for algorithms with coding techniques. We assess index quality by measuring both search efficiency and accuracy for the query set. Search efficiency is quantified in Queries Per Second (*QPS*), representing the number of queries processed per second. Search accuracy is measured through *Recall* and the *average distance ratio* (*ADR*). *Recall* is defined as $Recall = \frac{|G \cup S|}{k}$, where G is the ground truth set of the k nearest vectors, and S denotes the set of k results returned by the algorithm, with k set to 1 by default. *ADR*, defined in [86, 95], represents the average of the distance ratios between the retrieved k database vectors and the ground truth nearest vectors. All experimental results are derived from three repeated executions.

4.1.5 ***Environment Configuration.*** The experiments are conducted on a Linux server with dual Intel Xeon E5-2620 v3 CPUs (2.40GHz, 24 logical processors). The server employs an x86_64 architecture with multiple cache levels (32KB L1, 256KB L2, and 15MB L3) and 378GB RAM. All methods are implemented in C++ using intrinsics for SIMD access, compiled with g++ 9.3.1 using the *-Ofast* and *-march=native* options. SSE instructions are used by default for SIMD acceleration, and we also compare SSE (128-bit), AVX (256-bit), and AVX512 (512-bit)[4]. The Eigen library [1] optimizes matrix manipulations, while OpenMP facilitates parallel index construction and search execution across 24 threads.

4.2 Indexing Performance

The indexing times and index sizes for different methods are presented in Figures 6 and 7, respectively. The indexing time for HNSW reflects only the graph index construction, while other methods include extra coding preprocessing. The results show that optimized methods exhibit shorter indexing times than the original HNSW, indicating that compact coding techniques effectively accelerate HNSW construction. HNSW-Flash consistently outperforms other methods across almost all datasets, achieving speedups of 10.4× to 22.9×, highlighting `Flash`'s superiority in expediting index construction. Among baseline methods, HNSW-PCA and HNSW-SQ yield less than 2× speedup on most datasets, whereas HNSW-PQ demonstrates a better speedup ratio. Figure 7 illustrates that all optimized methods decrease the index size compared to the original HNSW by replacing high-dimensional vectors with compact vector codes. HNSW-PQ achieves the highest compression ratio, explaining its superior speedup in index construction. HNSW-Flash's memory layout optimization, which stores neighbor codewords alongside IDs, results in a lower compression ratio than HNSW-PQ; however, it reaches a better speedup due to efficient memory

[4]A new server is used for this evaluation due to the default does not support AVX512.

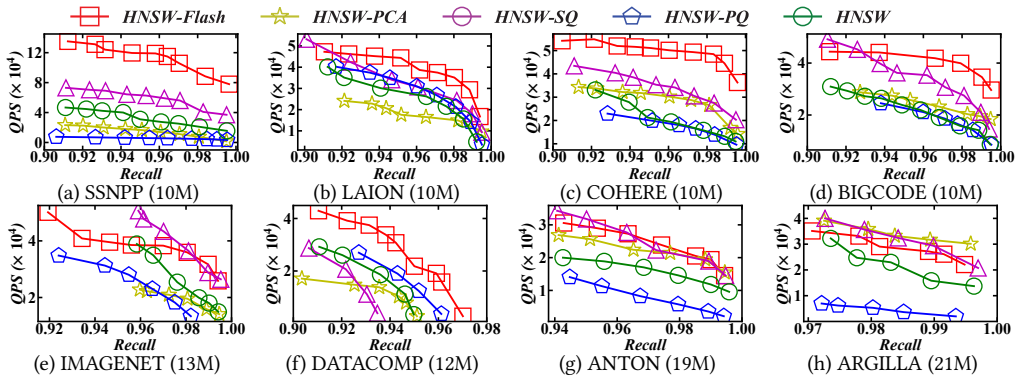


Fig. 8. Comparison of search performance across eight datasets (the top right indicates better performance).

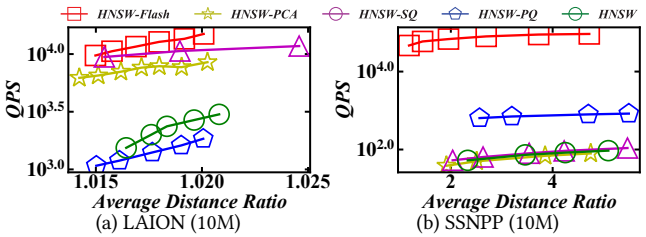
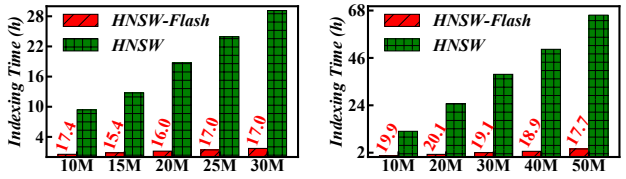


Fig. 9. The *QPS-ADR* curves (the top left is better).

access and arithmetic operations. Notably, HNSW-PQ suffers from a significant reduction in search performance (Figure 8) due to its high compression error, indicating the poor index quality.

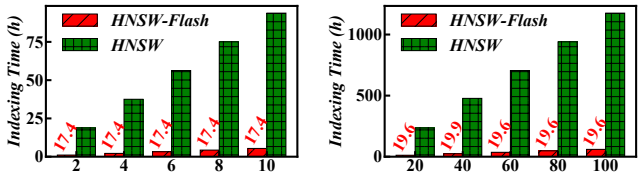
4.3 Search Performance

Figure 8 presents the *QPS-Recall* curves for all evaluated datasets, while Figure 9 shows the *QPS-ADR* curves for two representative datasets, with analogous trends observed for the remaining datasets. The results indicate that HNSW-Flash consistently outperforms other methods across nearly all datasets, demonstrating its ability to accelerate index construction without compromising search performance. Additionally, the optimizations in the Candidate Acquisition (CA) stage can be seamlessly integrated into the search procedure, as it shares similar steps with the CA process. The search performance of baseline methods varies significantly across datasets. For instance, HNSW-PCA achieves the best performance on ARGILLA but the worst on LAION, indicating sensitivity to data distribution. While HNSW-PQ offers considerable speedup in index construction, its search performance is inferior to standard HNSW due to degraded index quality. HNSW-SQ shows consistent search performance gains on most datasets, aligning with prior studies that optimizes search procedure using SQ [29]. However, HNSW-SQ provides limited indexing speedup, suggesting that a compact coding method suitable for search may not be ideal for index construction. This occurs because index construction involves an additional Neighbor Selection (NS) step and is therefore more complex than the search process. In addition, different methods may exhibit varying search performance rankings regarding *Recall* and *ADR* for the same dataset, indicating their false positive results are obviously different. Notably, `Flash` demonstrates substantial advantages in retrieving results closer to the ground truth vectors.



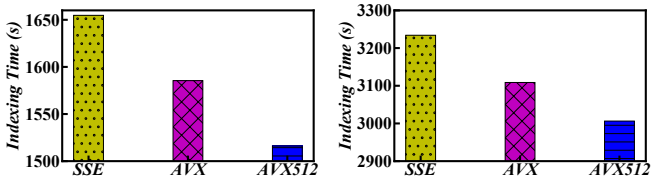
(a) LAION-30M (b) SSNPP-50M

Fig. 10. Scalability over different data volumes.



(a) LAION-100M (b) SSNPP-1B

Fig. 11. Scalability over different segment counts.



(a) LAION (10M) (b) SSNPP (10M)

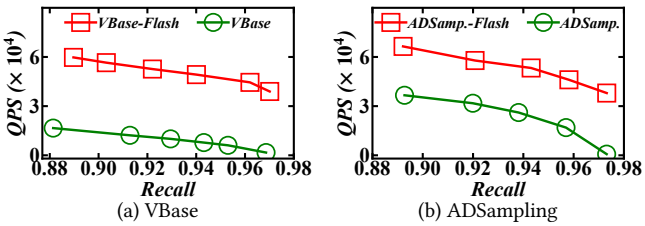
Fig. 12. Generality across different SIMD instruction sets.

4.4 Scalability

We evaluate the scalability of HNSW-Flash regarding data volume and the number of data segments, benchmarking its performance against standard HNSW. Indexing times for both HNSW and HNSW-Flash are assessed on the LAION-100M and SSNPP-1B datasets. Figure 10 presents indexing times across varying data volumes, with the data size within a single segment progressively increasing, while Figure 11 depicts indexing times for different segment counts, maintaining consistent data size per segment. For multi-segment evaluation, we accumulate the indexing times of each segment. The results demonstrate that HNSW-Flash significantly outperforms HNSW across all data volumes and segment counts. This highlights HNSW-Flash's superiority, particularly with larger datasets where HNSW incurs considerably higher indexing times. Consequently, HNSW-Flash achieves a notable reduction in absolute indexing times. In addition, HNSW-Flash can be seamlessly integrated into current distributed systems to expedite index construction within each segment, thereby providing cumulative acceleration.

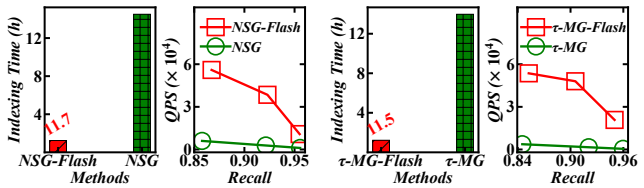
4.5 Generality

4.5.1 ***SIMD Instructions***. We integrate `Flash` into various SIMD instruction sets with differing register sizes to confirm its generality. We highlight `Flash` can be extended to 256-bit and 512-bit registers without altering its fundamental design principle. As shown in Figure 12, more advanced SIMD instructions with larger registers exhibit faster indexing processes. This is because SIMD instructions with larger registers can process more asymmetric distance tables (ADTs) per operation, resulting in more efficient arithmetic operations compared to those with smaller registers.



(a) VBase (b) ADSampling

Fig. 13. Generality across different implementations.



(a) NSG (b) τ-MG

Fig. 14. Generality across different graph algorithms.

Table 2. L1 cache misses before and after optimization.

	SSNPP	LAION	COHE.	BIGC.	IMAGE.	DATAC.	ANTON	ARGIL.
w/o opt.	19.08%	24.20%	24.97%	25.09%	23.02%	24.03%	25.92%	25.98%
w. opt.	5.21%	7.90%	6.17%	6.72%	7.38%	7.05%	5.81%	4.86%

Notably, the acceleration gained from larger registers is not strictly linear. Other factors, such as memory access patterns, also significantly impact indexing time. Moreover, the specific latency and throughput of instructions (such as register load times) vary across different SIMD instruction sets [22], further affecting the overall speedup.

4.5.2 ***Optimized HNSW Implementations.*** We apply `Flash` to optimized HNSW implementations, ADSampling [50] and VBase [121]. These optimizations retain the standard HNSW construction process, enabling `Flash` to directly accelerate indexing, as shown in Figure 6. Figure 13 presents the search performance of these implementations on LAION-10M before and after integrating `Flash`, demonstrating that `Flash` further improves search efficiency. This improvement stems from the fact that ADSampling's enhancements to distance comparisons and VBase's adjustments to the termination condition are orthogonal to `Flash`.

4.5.3 ***Graph Algorithms.*** We apply `Flash` to two other representative graph algorithms, NSG [49] and τ-MG [88]. Figure 14 presents the indexing time and search performance on LAION-10M. The results indicate that `Flash` significantly accelerates the indexing process of NSG and τ-MG while improving their search performance. These findings are consistent with those observed in HNSW. Both algorithms and HNSW share two key components—Candidate Acquisition (CA) and Neighbor Selection (NS)—in their indexing processes. Consequently, enhancing the CA and NS steps accelerates the indexing procedure across these graph algorithms.

4.6 Ablation Study

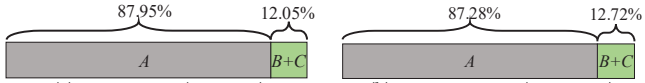
4.6.1 ***Memory Accesses.*** We utilize the *perf* tool to record CPU hardware counters for data reads and cache hits. Table 2 shows the L1 cache misses during index construction before and after our optimization across eight datasets. Each evaluation is conducted on one million samples per dataset, maintaining consistent indexing parameters. The results indicate a consistent reduction in

Table 3. Indexing time without and with SIMD optimization.

	SSNPP	LAION	COHE.	BIGC.	IMAGE.	DATAC.	ANTON	ARGIL.
w/o opt. (s)	233	154	270	214	88	154	157	140
w. opt. (s)	129	131	164	147	84	135	138	111

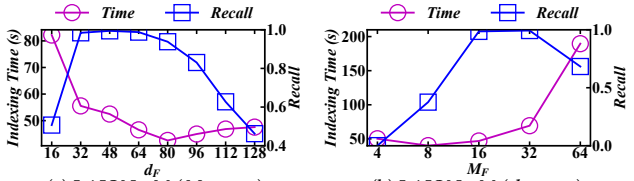
Table 4. Coding time (CT) and total indexing time (TIT).

	SSNPP	LAION	COHE.	BIGC.	IMAGE.	DATAC.	ANTON	ARGIL.
CT (h)	0.016	0.049	0.049	0.048	0.060	0.087	0.165	0.164
TIT (h)	0.599	0.542	0.668	0.619	0.487	0.730	1.035	1.083



(a) LAION-1M (D = 768) (b) ARGILLA-1M (D = 1024)

Fig. 15. Profiling of graph index construction time in HNSW-Flash. The distance computation time (B+C) consists of memory accesses (B) and arithmetic operations (C). A is the time of other tasks, such as data structure maintenance.



(a) LAION-1M (M_F = 16) (b) LAION-1M (d_F = 64)

Fig. 16. Effect of parameters within Flash.

cache misses across all eight datasets with Flash, demonstrating its efficacy in accelerating index construction by minimizing random memory accesses. Given that cache access is approximately 100 times faster than RAM access, even minor reductions in cache misses can significantly enhance the speed of distance computations [42].

4.6.2 ***Arithmetic Operations.*** To assess the impact of SIMD optimization, we compare indexing times with and without this option in Table 3, using the same dataset settings as described in Section 4.6.1. The results indicate that SIMD optimization can reduce indexing time by up to 45%, highlighting its effectiveness in accelerating index construction through efficient arithmetic operations. Notably, the indexing times in Table 3 include vector coding time, which constitutes approximately 10% of the total indexing time; however, SIMD optimization does not affect the coding time.

4.6.3 ***Profiling of indexing time.*** Table 4 presents the coding time (CT) for Flash alongside the total indexing time (TIT) for HNSW-Flash, with TIT encompassing both CT and the graph index construction time (GIT). Data scales for all datasets align with those in Figure 6. The results show that CT constitutes only 10% of TIT, demonstrating that Flash requires little processing time. We further profile GIT using the *perf* tool in Figure 15. The results indicate that distance computation occupies a small fraction of GIT for HNSW-Flash. This suggests the primary bottlenecks associated with memory accesses and arithmetic operations in HNSW have been removed through the integration of Flash.

4.7 Parameter Sensitivity

HNSW-Flash has two adjustable parameters to balance construction efficiency and index quality: the dimensions of principal components (d_F) and the number of subspaces (M_F). Other parameters, like bits per subspace (L_F) are fixed to align with index features and hardware constraints. Figure 16 illustrates indexing time and search accuracy across varying d_F and M_F, using the recall rate at a given latency as a measure of index quality. In Figure 16(a), with M_F fixed at 16, index quality improves with increasing d_F initially but subsequently declines beyond a certain threshold. Correspondingly, indexing time decreases initially, then rises after reaching a critical point. This phenomenon arises from potential information loss at lower dimensions and redundancy at higher dimensions, given the fixed L_F. Notably, the optimal points for indexing time and search accuracy occur at different d_F, reflecting the distinct sensitivities of index construction and search procedures to information content. While the search focuses on nearest neighbors, index construction requires navigable neighbors, even if more distant [49, 78]. In Figure 16(b), with d_F set to 64, increasing M_F extends indexing time. The search accuracy improves initially but later declines due to increased computational overhead and additional register loads.

5 Discussion

Based on our research, we highlight three notable findings:

(1) A compact coding method that significantly enhances search performance may not be suitable for index construction. Recent research [66, 75, 113, 116] has integrated compact coding techniques, such as PQ [67], into the search process while preserving the original HNSW index construction. This approach improves search performance but incurs additional indexing time. Note that the index construction process presents greater challenges due to its more complex execution logic relative to the search procedure [78, 104]. Consequently, directly applying existing compact coding methods to HNSW construction may not yield substantial improvements in indexing speed. Our empirical results indicate that while HNSW-SQ enhances search performance compared to standard HNSW, it provides only marginal improvements in indexing speed.

(2) Reducing more dimensions may bring higher accuracy. While it might seem intuitive that preserving more dimensions during vector dimensionality reduction would increase accuracy [50, 88, 113], focusing on principal components is advisable when vector representation is constrained by limited bit allocation. This approach reduces the impact of less significant components, which tend to suffer from poor bit usage. For instance, `Flash` leverages lower-dimensional principal components, significantly accelerating index construction and delivering superior search performance compared to higher-dimensional counterparts (Figure 16).

(3) Encoding vectors and distances with a tiny amount of bits to align with hardware constraints may yield substantial benefits. In compact coding methods, fewer bits typically enhance efficiency but may compromise accuracy, while more bits can improve accuracy at the cost of efficiency [52, 67]. Therefore, determining an optimal bit count is essential for balancing accuracy and efficiency. However, this optimal count may not align with hardware constraints. Addressing this requires redefining the balance to accommodate hardware specifications. With tailored hardware optimizations, using fewer bits may achieve a better balance than the original optimal counts. For example, `Flash` encodes distances with 8 bits to leverage SIMD capabilities, achieving a more favorable balance than the standard 32-bit configuration.

6 Conclusion

In this paper, we investigate the index construction efficiency of graph-based ANNS methods on modern CPUs, using the representative HNSW index. We identify that low construction efficiency

arises from high memory access latency and suboptimal arithmetic operation efficiency in distance computations. We develop three baseline methods incorporating existing compact coding techniques into the HNSW construction process, yielding valuable lessons. This leads to the design of `Flash`, tailored for the HNSW construction process. `Flash` reduces random memory accesses and leverages SIMD instructions, improving cache hits and arithmetic operations. Extensive evaluations confirm `Flash`'s superiority in accelerating graph indexing while maintaining or enhancing search performance. We also apply `Flash` to other graph indexes, optimized HNSW implementations, and various SIMD instruction sets to verify its generality.

Acknowledgments

This work was supported in part by the NSFC under Grants No. (62025206 and U23A20296), Zhejiang Province's "Lingyan" R&D Project under Grant No. 2024C01259, CCF-Aliyun2024004, Ningbo Science and Technology Special Projects under Grant No. 2023Z212, Ningbo Yongjiang Talent Introduction Programme (2022A-237-G), and the Key Lab of Intelligent Computing Based Big Data of Zhejiang Province. Yunjun Gao is the corresponding author of this work.

References

[1] 2009. Eigen is a C++ template library for linear algebra: matrices, vectors, numerical solvers, and related algorithms. https://eigen.tuxfamily.org/index.php?title=Main_Page. [Online; accessed 01-June-2024].
[2] 2017. Header-only C++/python library for fast approximate nearest neighbors. https://github.com/nmslib/hnswlib. [Online; accessed 25-July-2024].
[3] 2017. A Library for Efficient Similarity Search and Clustering of Dense Vectors. https://github.com/facebookresearch/faiss. [Online; accessed 25-July-2024].
[4] 2017. TOROS N2 - lightweight approximate Nearest Neighbor library which runs fast even with large datasets. https://github.com/kakao/n2. [Online; accessed 25-July-2024].
[5] 2018. Possible to continue setting data after building index? https://github.com/nmslib/nmslib/issues/73. [Online; accessed 20-September-2024].
[6] 2021. Billion-Scale Approximate Nearest Neighbor Search Challenge: NeurIPS'21 competition track. https://big-ann-benchmarks.com/neurips21.html. [Online; accessed 01-June-2024].
[7] 2023. Cohere/wikipedia-22-12-es-embeddings. https://huggingface.co/datasets/Cohere/wikipedia-22-12-es-embeddings. [Online; accessed 23-September-2024].
[8] 2023. kinianlo/imagenet_embeddings. https://huggingface.co/datasets/kinianlo/imagenet_embeddings. [Online; accessed 23-September-2024].
[9] 2023. nielsr/datacomp-small-with-embeddings-and-cluster-labels. https://huggingface.co/datasets/nielsr/datacomp-small-with-embeddings-and-cluster-labels. [Online; accessed 23-September-2024].
[10] 2023. Scalar Quantization: Background, Practices & More | Qdrant. https://qdrant.tech/articles/scalar-quantization. [Online; accessed 02-August-2024].
[11] 2023. Vector search in Elasticsearch: The rationale behind the design. https://www.elastic.co/search-labs/blog/vector-search-elasticsearch-rationale. [Online; accessed 14-July-2024].
[12] 2024. Accepted Research Papers in ICDE'24. https://icde2024.github.io/papers.html. [Online; accessed 14-July-2024].
[13] 2024. Accepted Research Papers in SIGMOD'24. https://2024.sigmod.org/sigmod-list.html. [Online; accessed 14-July-2024].
[14] 2024. Accepted Research Papers in VLDB'24. https://vldb.org/pvldb/volumes/17. [Online; accessed 14-July-2024].
[15] 2024. anton-l/wiki-embed-mxbai-embed-large-v1. https://huggingface.co/datasets/anton-l/wiki-embed-mxbai-embed-large-v1. [Online; accessed 21-September-2024].
[16] 2024. argilla-warehouse/personahub-fineweb-edu-4-embeddings. https://huggingface.co/datasets/argilla-warehouse/personahub-fineweb-edu-4-embeddings. [Online; accessed 23-September-2024].
[17] 2024. bigcode/stack-exchange-embeddings-20230914. https://huggingface.co/datasets/bigcode/stack-exchange-embeddings-20230914. [Online; accessed 23-September-2024].
[18] 2024. Hierarchical Navigable Small Worlds (HNSW). https://www.pinecone.io/learn/series/faiss/hnsw/. [Online; accessed 14-July-2024].
[19] 2024. hnsqlite. https://github.com/jiggy-ai/hnsqlite. [Online; accessed 20-August-2024].
[20] 2024. Hugging Face. https://huggingface.co/datasets. [Online; accessed 01-June-2024].

[21] 2024. Intel® Intrinsics Guide. https://www.intel.com/content/www/us/en/docs/intrinsics-guide/index.html#text=shuffle_epi32&ig_expand=6003,6000,5997,6000,5999,5998. [Online; accessed 23-September-2024].
[22] 2024. Intel® Intrinsics Guide. https://www.intel.com/content/www/us/en/docs/intrinsics-guide/index.html#techs=AVX_ALL,AVX_512&text=loadu&ig_expand=4108,4110. [Online; accessed 15-December-2024].
[23] 2024. Knowhere. https://milvus.io/docs/knowhere.md. [Online; accessed 01-September-2024].
[24] 2024. The LAION2B and projections. https://sisap-challenges.github.io/2024/datasets/. [Online; accessed 23-September-2024].
[25] 2024. New embedding models and API updates. https://openai.com/index/new-embedding-models-and-api-updates/. [Online; accessed 20-September-2024].
[26] 2024. Performance analysis tools based on Linux perf_events (aka perf) and ftrace. https://github.com/brendangregg/perf-tools. [Online; accessed 23-September-2024].
[27] 2024. Step-by-Step Guide to Choosing the Best Embedding Model for Your Application. https://weaviate.io/blog/how-to-choose-an-embedding-model. [Online; accessed 20-September-2024].
[28] 2024. What Goes Around Comes Around… And Around…. https://sigmodrecord.org/publications/sigmodRecord/2406/pdfs/04_Surveys_Stonebraker.pdf. [Online; accessed 16-July-2024].
[29] Cecilia Aguerrebere, Ishwar Singh Bhati, Mark Hildebrand, Mariano Tepper, and Theodore L. Willke. 2023. Similarity search in the blink of an eye with compressed indices. *Proceedings of the VLDB Endowment (PVLDB)* 16, 11 (2023), 3433–3446.
[30] Fabien André, Anne-Marie Kermarrec, and Nicolas Le Scouarnec. 2015. Cache locality is not enough: High-Performance Nearest Neighbor Search with Product Quantization Fast Scan. *Proceedings of the VLDB Endowment (PVLDB)* 9, 4 (2015), 288–299.
[31] Fabien André, Anne-Marie Kermarrec, and Nicolas Le Scouarnec. 2017. Accelerated Nearest Neighbor Search with Quick ADC. In *Proceedings of the 2017 ACM on International Conference on Multimedia Retrieval (ICMR)*. 159–166.
[32] Akhil Arora, Sakshi Sinha, Piyush Kumar, and Arnab Bhattacharya. 2018. HD-Index: Pushing the Scalability-Accuracy Boundary for Approximate kNN Search in High-Dimensional Spaces. *Proceedings of the VLDB Endowment (PVLDB)* 11, 8 (2018), 906–919.
[33] Martin Aumüller and Matteo Ceccarello. 2023. Recent Approaches and Trends in Approximate Nearest Neighbor Search, with Remarks on Benchmarking. *IEEE Data Engineering Bulletin* 46, 3 (2023), 89–105.
[34] Ilias Azizi, Karima Echihabi, and Themis Palpanas. 2023. Elpis: Graph-Based Similarity Search for Scalable Data Science. *Proceedings of the VLDB Endowment (PVLDB)* 16, 6 (2023), 1548–1559.
[35] Dmitry Baranchuk, Dmitry Persiyanov, Anton Sinitsin, and Artem Babenko. 2019. Learning to Route in Similarity Graphs. In *Proceedings of the 36th International Conference on Machine Learning(ICML)*, Vol. 97. 475–484.
[36] Petros Boufounos. 2012. Universal Rate-Efficient Scalar Quantization. *IEEE Transactions on Information Theory* 58, 3 (2012), 1861–1872.
[37] Cheng Chen, Chenzhe Jin, Yunan Zhang, Sasha Podolsky, Chun Wu, Szu-Po Wang, Eric Hanson, Zhou Sun, Robert Walzer, and Jianguo Wang. 2024. SingleStore-V: An Integrated Vector Database System in SingleStore. *Proceedings of the VLDB Endowment (PVLDB)* 17, 12 (2024), 3772–3785.
[38] Meng Chen, Kai Zhang, Zhenying He, Yinan Jing, and X. Sean Wang. 2024. RoarGraph: A Projected Bipartite Graph for Efficient Cross-Modal Approximate Nearest Neighbor Search. *Proceedings of the VLDB Endowment (PVLDB)* 17, 11 (2024), 2735–2749.
[39] Patrick H. Chen, Wei-Cheng Chang, Jyun-Yu Jiang, Hsiang-Fu Yu, Inderjit S. Dhillon, and Cho-Jui Hsieh. 2023. FINGER: Fast Inference for Graph-based Approximate Nearest Neighbor Search. In *Proceedings of the ACM Web Conference (WWW)*. 3225–3235.
[40] Qi Chen, Bing Zhao, Haidong Wang, Mingqin Li, Chuanjie Liu, Zengzhong Li, Mao Yang, and Jingdong Wang. 2021. SPANN: Highly-efficient Billion-scale Approximate Nearest Neighborhood Search. In *Advances in Neural Information Processing Systems 34: Annual Conference on Neural Information Processing Systems (NeurIPS)*. 5199–5212.
[41] Rihan Chen, Bin Liu, Han Zhu, Yaoxuan Wang, Qi Li, Buting Ma, Qingbo Hua, Jun Jiang, Yunlong Xu, Hongbo Deng, and Bo Zheng. 2022. Approximate Nearest Neighbor Search under Neural Similarity Metric for Large-Scale Recommendation. In *Proceedings of the 31st ACM International Conference on Information & Knowledge Management (CIKM)*. 3013–3022.
[42] Benjamin Coleman, Santiago Segarra, Alexander J. Smola, and Anshumali Shrivastava. 2022. Graph Reordering for Cache-Efficient Near Neighbor Search. In *Advances in Neural Information Processing Systems 35: Annual Conference on Neural Information Processing Systems (NeurIPS)*.
[43] Shiyuan Deng, Xiao Yan, KW Ng Kelvin, Chenyu Jiang, and James Cheng. 2019. Pyramid: A general framework for distributed similarity search on large-scale datasets. In *IEEE International Conference on Big Data*. 1066–1071.
[44] Wei Dong, Moses Charikar, and Kai Li. 2011. Efficient K-nearest Neighbor Graph Construction for Generic Similarity Measures. In *Proceedings of the 20th International Conference on World Wide Web (WWW)*. 577–586.

[45] Ishita Doshi, Dhritiman Das, Ashish Bhutani, Rajeev Kumar, Rushi Bhatt, and Niranjan Balasubramanian. 2022. LANNS: A Web-Scale Approximate Nearest Neighbor Lookup System. *Proceedings of the VLDB Endowment (PVLDB)* 15, 4 (2022), 850–858.

[46] Matthijs Douze, Alexandr Guzhva, Chengqi Deng, Jeff Johnson, Gergely Szilvasy, Pierre-Emmanuel Mazaré, Maria Lomeli, Lucas Hosseini, and Hervé Jégou. 2024. The faiss library. *arXiv:2401.08281* (2024).

[47] Cole Foster and Benjamin B. Kimia. 2023. Computational Enhancements of HNSW Targeted to Very Large Datasets. In *Similarity Search and Applications - 16th International Conference (SISAP)*, Vol. 14289. 291–299.

[48] Cong Fu, Changxu Wang, and Deng Cai. 2021. High Dimensional Similarity Search with Satellite System Graph: Efficiency, Scalability, and Unindexed Query Compatibility. *IEEE Transactions on Pattern Analysis and Machine Intelligence (TPAMI)* 44, 8 (2021), 4139–4150.

[49] Cong Fu, Chao Xiang, Changxu Wang, and Deng Cai. 2019. Fast Approximate Nearest Neighbor Search With The Navigating Spreading-out Graph. *Proceedings of the VLDB Endowment (PVLDB)* 12, 5 (2019), 461–474.

[50] Jianyang Gao and Cheng Long. 2023. High-Dimensional Approximate Nearest Neighbor Search: with Reliable and Efficient Distance Comparison Operations. *Proceedings of the ACM on Management of Data (PACMMOD)* 1, 2 (2023), 137:1–137:27.

[51] Jianyang Gao and Cheng Long. 2024. RaBitQ: Quantizing High-Dimensional Vectors with a Theoretical Error Bound for Approximate Nearest Neighbor Search. *Proceedings of the ACM on Management of Data (PACMMOD)* 2, 3 (2024), 167:1–167:27.

[52] Tiezheng Ge, Kaiming He, Qifa Ke, and Jian Sun. 2013. Optimized Product Quantization for Approximate Nearest Neighbor Search. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. 2946–2953.

[53] Siddharth Gollapudi, Neel Karia, Varun Sivashankar, Ravishankar Krishnaswamy, Nikit Begwani, Swapnil Raz, Yiyong Lin, Yin Zhang, Neelam Mahapatro, Premkumar Srinivasan, et al. 2023. Filtered-DiskANN: Graph Algorithms for Approximate Nearest Neighbor Search with Filters. In *Proceedings of the ACM Web Conference (WWW)*. 3406–3416.

[54] Qian-wen Gou, Yunwei Dong, YuJiao Wu, and Qiao Ke. 2024. Semantic similarity-based program retrieval: a multi-relational graph perspective. *Frontiers of Computer Science (FCS)* 18, 3 (2024), 183209.

[55] Fabian Groh, Lukas Ruppert, Patrick Wieschollek, and Hendrik PA Lensch. 2022. Ggnn: Graph-based gpu nearest neighbor search. *IEEE Transactions on Big Data (TBD)* 9, 1 (2022), 267–279.

[56] Rentong Guo, Xiaofan Luan, Long Xiang, Xiao Yan, Xiaomeng Yi, Jigao Luo, Qianya Cheng, Weizhi Xu, Jiarui Luo, Frank Liu, Zhenshan Cao, Yanliang Qiao, Ting Wang, Bo Tang, and Charles Xie. 2022. Manu: A Cloud Native Vector Database Management System. *Proceedings of the VLDB Endowment (PVLDB)* 15, 12 (2022), 3548–3561.

[57] Ruiqi Guo, Philip Sun, Erik Lindgren, Quan Geng, David Simcha, Felix Chern, and Sanjiv Kumar. 2020. Accelerating Large-Scale Inference with Anisotropic Vector Quantization. In *Proceedings of the 37th International Conference on Machine Learning (ICML)*. 3887–3896.

[58] Peng Han, Silin Zhou, Jie Yu, Zichen Xu, Lisi Chen, and Shuo Shang. 2023. Personalized Re-ranking for Recommendation with Mask Pretraining. *Data Science and Engineering (DSE)* 8, 4 (2023), 357–367.

[59] Junxian He, Graham Neubig, and Taylor Berg-Kirkpatrick. 2021. Efficient Nearest Neighbor Language Models. In *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing (EMNLP)*. 5703–5714.

[60] Qiang Huang, Jianlin Feng, Yikai Zhang, Qiong Fang, and Wilfred Ng. 2015. Query-Aware Locality-Sensitive Hashing for Approximate Nearest Neighbor Search. *Proceedings of the VLDB Endowment (PVLDB)* 9, 1 (2015), 1–12.

[61] Yuchen Huang, Xiaopeng Fan, Song Yan, and Chuliang Weng. 2024. Neos: A NVMe-GPUs Direct Vector Service Buffer in User Space. In *IEEE 40th International Conference on Data Engineering (ICDE)*. 3767–3781.

[62] Piotr Indyk and Haike Xu. 2023. Worst-case Performance of Popular Approximate Nearest Neighbor Search Implementations: Guarantees and Limitations. In *Advances in Neural Information Processing Systems 36: Annual Conference on Neural Information Processing Systems (NeurIPS)*.

[63] Keita Iwabuchi, Trevor Steil, Benjamin Priest, Roger Pearce, and Geoffrey Sanders. 2023. Towards A Massive-scale Distributed Neighborhood Graph Construction. In *Proceedings of the SC'23 Workshops of The International Conference on High Performance Computing, Network, Storage, and Analysis (SC Workshops)*. 728–738.

[64] Masajiro Iwasaki and Daisuke Miyazaki. 2018. Optimization of Indexing Based on k-Nearest Neighbor Graph for Proximity Search in High-dimensional Data. *arXiv:1810.07355* (2018).

[65] Junhyeok Jang, Hanjin Choi, Hanyeoreum Bae, Seungjun Lee, Miryeong Kwon, and Myoungsoo Jung. 2023. CXL-ANNS: Software-Hardware Collaborative Memory Disaggregation and Computation for Billion-Scale Approximate Nearest Neighbor Search. In *Proceedings of the 2023 USENIX Annual Technical Conference (USENIX ATC)*. 585–600.

[66] Suhas Jayaram Subramanya, Fnu Devvrit, Harsha Vardhan Simhadri, Ravishankar Krishnawamy, and Rohan Kadekodi. 2019. DiskANN: Fast Accurate Billion-point Nearest Neighbor Search on a Single Node. In *Advances in Neural Information Processing Systems 32: Annual Conference on Neural Information Processing Systems (NeurIPS)*. 13748–13758.

[67] Hervé Jégou, Matthijs Douze, and Cordelia Schmid. 2011. Product Quantization for Nearest Neighbor Search. *IEEE Transactions on Pattern Analysis and Machine Intelligence (TPAMI)* 33, 1 (2011), 117–128.

[68] Wenqi Jiang, Hang Hu, Torsten Hoefler, and Gustavo Alonso. 2024. Accelerating Graph-based Vector Search via Delayed-Synchronization Traversal. *arXiv:2406.12385* (2024).

[69] Ian T Jolliffe and Jorge Cadima. 2016. Principal component analysis: a review and recent developments. *Philosophical transactions of the royal society A: Mathematical, Physical and Engineering Sciences* 374, 2065 (2016), 20150202.

[70] Omar Shahbaz Khan, Martin Aumüller, and Björn Þór Jónsson. 2023. Suitability of Nearest Neighbour Indexes for Multimedia Relevance Feedback. In *Similarity Search and Applications - 16th International Conference (SISAP)*. 133–147.

[71] Conglong Li, Minjia Zhang, David G. Andersen, and Yuxiong He. 2020. Improving Approximate Nearest Neighbor Search through Learned Adaptive Early Termination. In *Proceedings of the International Conference on Management of Data (SIGMOD)*. 2539–2554.

[72] Jie Li, Haifeng Liu, Chuanghua Gui, Jianyu Chen, Zhenyuan Ni, Ning Wang, and Yuan Chen. 2018. The Design and Implementation of a Real Time Visual Search System on JD E-commerce Platform. In *Proceedings of the 19th International Middleware Conference (Middleware)*. 9–16.

[73] Wen Li, Ying Zhang, Yifang Sun, Wei Wang, Mingjie Li, Wenjie Zhang, and Xuemin Lin. 2020. Approximate Nearest Neighbor Search on High Dimensional Data - Experiments, Analyses, and Improvement. *IEEE Transactions on Knowledge and Data Engineering (TKDE)* 32, 8 (2020), 1475–1488.

[74] Jun Liu, Zhenhua Zhu, Jingbo Hu, Hanbo Sun, Li Liu, Lingzhi Liu, Guohao Dai, Huazhong Yang, and Yu Wang. 2022. Optimizing Graph-based Approximate Nearest Neighbor Search: Stronger and Smarter. In *23rd IEEE International Conference on Mobile Data Management (MDM)*. 179–184.

[75] Kejing Lu, Mineichi Kudo, Chuan Xiao, and Yoshiharu Ishikawa. 2022. HVS: hierarchical graph structure based on voronoi diagrams for solving approximate nearest neighbor search. *Proceedings of the VLDB Endowment (PVLDB)* 15, 2 (2022), 246–258.

[76] Kejing Lu, Hongya Wang, Wei Wang, and Mineichi Kudo. 2020. VHP: Approximate Nearest Neighbor Search via Virtual Hypersphere Partitioning. *Proceedings of the VLDB Endowment (PVLDB)* 13, 9 (2020), 1443–1455.

[77] Kejing Lu, Chuan Xiao, and Yoshiharu Ishikawa. 2024. Probabilistic Routing for Graph-Based Approximate Nearest Neighbor Search. *arXiv:2402.11354* (2024).

[78] Yury A. Malkov and D. A. Yashunin. 2020. Efficient and Robust Approximate Nearest Neighbor Search Using Hierarchical Navigable Small World Graphs. *IEEE Transactions on Pattern Analysis and Machine Intelligence (TPAMI)* 42, 4 (2020), 824–836.

[79] Xupeng Miao, Hailin Zhang, Yining Shi, Xiaonan Nie, Zhi Yang, Yangyu Tao, and Bin Cui. 2021. HET: Scaling out Huge Embedding Model Training via Cache-enabled Distributed Framework. *Proceedings of the VLDB Endowment (PVLDB)* 15, 2 (2021), 312–320.

[80] Javier Alvaro Vargas Muñoz, Zanoni Dias, and Ricardo da Silva Torres. 2019. A Genetic Programming Approach for Searching on Nearest Neighbors Graphs. In *Proceedings of the International Conference on Multimedia Retrieval (ICMR)*. 43–47.

[81] Javier Alvaro Vargas Muñoz, Marcos André Gonçalves, Zanoni Dias, and Ricardo da Silva Torres. 2019. Hierarchical Clustering-Based Graphs for Large Scale Approximate Nearest Neighbor Search. *Pattern Recognition (PR)* 96 (2019).

[82] Hiroyuki Ootomo, Akira Naruse, Corey Nolet, Ray Wang, Tamas Feher, and Yong Wang. 2024. CAGRA: Highly Parallel Graph Construction and Approximate Nearest Neighbor Search for GPUs. In *IEEE 40th International Conference on Data Engineering (ICDE)*. 4236–4247.

[83] James Jie Pan, Jianguo Wang, , and Guoliang Li. 2024. Vector Database Management Techniques and Systems. In *Proceedings of the International Conference on Management of Data (SIGMOD), Tutorial Track*.

[84] James Jie Pan, Jianguo Wang, and Guoliang Li. 2024. Survey of vector database management systems. *VLDB J.* 33, 5 (2024), 1591–1615.

[85] Liana Patel, Peter Kraft, Carlos Guestrin, and Matei Zaharia. 2024. ACORN: Performant and Predicate-Agnostic Search Over Vector Embeddings and Structured Data. *Proceedings of the ACM on Management of Data (PACMMOD)* 2, 3 (2024), 120.

[86] Marco Patella and Paolo Ciaccia. 2008. The Many Facets of Approximate Similarity Search. In *First International Workshop on Similarity Search and Applications (SISAP)*. 10–21.

[87] Hongwu Peng, Shiyang Chen, Zhepeng Wang, Junhuan Yang, Scott A. Weitze, Tong Geng, Ang Li, Jinbo Bi, Minghu Song, Weiwen Jiang, Hang Liu, and Caiwen Ding. 2021. Optimizing FPGA-based Accelerator Design for Large-Scale Molecular Similarity Search (Special Session Paper). In *IEEE/ACM International Conference On Computer Aided Design (ICCAD)*. 1–7.

[88] Yun Peng, Byron Choi, Tsz Nam Chan, Jianye Yang, and Jianliang Xu. 2023. Efficient Approximate Nearest Neighbor Search in Multi-dimensional Databases. *Proceedings of the ACM on Management of Data (PACMMOD)* 1, 1 (2023), 54:1–54:27.

[89] Zhen Peng, Minjia Zhang, Kai Li, Ruoming Jin, and Bin Ren. 2023. iQAN: Fast and Accurate Vector Search with Efficient Intra-Query Parallelism on Multi-Core Architectures. In *Proceedings of the 28th ACM SIGPLAN Annual Symposium on Principles and Practice of Parallel Programming (PPoPP)*. 313–328.

[90] Samira Pouyanfar, Saad Sadiq, Yilin Yan, Haiman Tian, Yudong Tao, Maria E. Presa Reyes, Mei-Ling Shyu, Shu-Ching Chen, and S. S. Iyengar. 2019. A Survey on Deep Learning: Algorithms, Techniques, and Applications. *ACM Computing Surveys (CSUR)* 51, 5 (2019), 92:1–92:36.

[91] Jie Ren, Minjia Zhang, and Dong Li. 2020. HM-ANN: Efficient Billion-Point Nearest Neighbor Search on Heterogeneous Memory. In *Advances in Neural Information Processing Systems 33: Annual Conference on Neural Information Processing Systems (NeurIPS)*.

[92] Xuanhua Shi, Zhigao Zheng, Yongluan Zhou, Hai Jin, Ligang He, Bo Liu, and Qiang-Sheng Hua. 2018. Graph Processing on GPUs: A Survey. *ACM Computing Surveys (CSUR)* 50, 6 (2018), 81:1–81:35.

[93] Aditi Singh, Suhas Jayaram Subramanya, Ravishankar Krishnaswamy, and Harsha Vardhan Simhadri. 2021. Freshdiskann: A fast and accurate graph-based ann index for streaming similarity search. *arXiv:2105.09613* (2021).

[94] Narayanan Sundaram, Aizana Turmukhametova, Nadathur Satish, Todd Mostak, Piotr Indyk, Samuel Madden, and Pradeep Dubey. 2013. Streaming Similarity Search over one Billion Tweets using Parallel Locality-Sensitive Hashing. *Proceedings of the VLDB Endowment (PVLDB)* 6, 14 (2013), 1930–1941.

[95] Yufei Tao, Ke Yi, Cheng Sheng, and Panos Kalnis. 2010. Efficient and accurate nearest neighbor and closest pair search in high-dimensional space. *TODS* 35, 3 (2010), 20:1–20:46.

[96] Mariano Tepper, Ishwar Singh Bhati, Cecilia Aguerrebere, Mark Hildebrand, and Ted Willke. 2023. LeanVec: Search your vectors faster by making them fit. *arXiv:2312.16335* (2023).

[97] Bing Tian, Haikun Liu, Zhuohui Duan, Xiaofei Liao, Hai Jin, and Yu Zhang. 2024. Scalable Billion-point Approximate Nearest Neighbor Search Using SmartSSDs. In *Proceedings of the 2024 USENIX Annual Technical Conference (USENIX ATC)*. 1135–1150.

[98] Hui Wang, Wan-Lei Zhao, Xiangxiang Zeng, and Jianye Yang. 2021. Fast k-NN Graph Construction by GPU based NN-Descent. In *The 30th ACM International Conference on Information and Knowledge Management (CIKM)*. 1929–1938.

[99] Jun Wang, Wei Liu, Sanjiv Kumar, and Shih-Fu Chang. 2016. Learning to Hash for Indexing Big Data - A Survey. *Proc. IEEE* 104, 1 (2016), 34–57.

[100] Jianguo Wang, Xiaomeng Yi, Rentong Guo, Hai Jin, Peng Xu, Shengjun Li, Xiangyu Wang, Xiangzhou Guo, Chengming Li, Xiaohai Xu, Kun Yu, Yuxing Yuan, Yinghao Zou, Jiquan Long, Yudong Cai, Zhenxiang Li, Zhifeng Zhang, Yihua Mo, Jun Gu, Ruiyi Jiang, Yi Wei, and Charles Xie. 2021. Milvus: A Purpose-Built Vector Data Management System. In *Proceedings of the International Conference on Management of Data (SIGMOD)*. 2614–2627.

[101] Mengzhao Wang, Lingwei Lv, Xiaoliang Xu, Yuxiang Wang, Qiang Yue, and Jiongkang Ni. 2022. Navigable Proximity Graph-Driven Native Hybrid Queries with Structured and Unstructured Constraints. *arXiv:2203.13601* (2022).

[102] Mengzhao Wang, Lingwei Lv, Xiaoliang Xu, Yuxiang Wang, Qiang Yue, and Jiongkang Ni. 2023. An Efficient and Robust Framework for Approximate Nearest Neighbor Search with Attribute Constraint. In *Advances in Neural Information Processing Systems 36: Annual Conference on Neural Information Processing Systems (NeurIPS)*. 15738–15751.

[103] Mengzhao Wang, Weizhi Xu, Xiaomeng Yi, Songlin Wu, Zhangyang Peng, Xiangyu Ke, Yunjun Gao, Xiaoliang Xu, Rentong Guo, and Charles Xie. 2024. Starling: An I/O-Efficient Disk-Resident Graph Index Framework for High-Dimensional Vector Similarity Search on Data Segment. *Proceedings of the ACM on Management of Data (PACMMOD)* 2, 1 (2024), 14:1–14:27.

[104] Mengzhao Wang, Xiaoliang Xu, Qiang Yue, and Yuxiang Wang. 2021. A Comprehensive Survey and Experimental Comparison of Graph-Based Approximate Nearest Neighbor Search. *Proceedings of the VLDB Endowment (PVLDB)* 14, 11 (2021), 1964–1978.

[105] Runhui Wang and Dong Deng. 2020. DeltaPQ: Lossless Product Quantization Code Compression for High Dimensional Similarity Search. *Proceedings of the VLDB Endowment (PVLDB)* 13, 13 (2020), 3603–3616.

[106] Yifan Wang, Haodi Ma, and Daisy Zhe Wang. 2022. LIDER: An Efficient High-dimensional Learned Index for Large-scale Dense Passage Retrieval. *Proceedings of the VLDB Endowment (PVLDB)* 16, 2 (2022), 154–166.

[107] Zeyu Wang, Haoran Xiong, Zhenying He, Peng Wang, et al. 2024. Distance Comparison Operators for Approximate Nearest Neighbor Search: Exploration and Benchmark. *arXiv:2403.13491* (2024).

[108] Chuangxian Wei, Bin Wu, Sheng Wang, Renjie Lou, Chaoqun Zhan, Feifei Li, and Yuanzhe Cai. 2020. AnalyticDB-V: A Hybrid Analytical Engine Towards Query Fusion for Structured and Unstructured Data. *Proceedings of the VLDB Endowment (PVLDB)* 13, 12 (2020), 3152–3165.

[109] Shitao Xiao, Zheng Liu, Weihao Han, Jianjin Zhang, Defu Lian, Yeyun Gong, Qi Chen, Fan Yang, Hao Sun, Yingxia Shao, and Xing Xie. 2022. Distill-VQ: Learning Retrieval Oriented Vector Quantization By Distilling Knowledge from Dense Embeddings. In *The 45th International ACM SIGIR Conference on Research and Development in Information Retrieval*. 1513–1523.

[110] Xiaoliang Xu, Mengzhao Wang, Yuxiang Wang, and Dingcheng Ma. 2021. Two-stage routing with optimized guided search and greedy algorithm on proximity graph. *Knowledge-Based Systems (KBS)* 229 (2021), 107305.
[111] Yuming Xu, Hengyu Liang, Jin Li, Shuotao Xu, Qi Chen, Qianxi Zhang, Cheng Li, Ziyue Yang, Fan Yang, Yuqing Yang, Peng Cheng, and Mao Yang. 2023. SPFresh: Incremental In-Place Update for Billion-Scale Vector Search. In *Proceedings of the 29th Symposium on Operating Systems Principles (SOSP)*. 545–561.
[112] Kaixiang Yang, Hongya Wang, Bo Xu, Wei Wang, Yingyuan Xiao, Ming Du, and Junfeng Zhou. 2021. Tao: A Learning Framework for Adaptive Nearest Neighbor Search using Static Features Only. *arXiv:2110.00696* (2021).
[113] Mingyu Yang, Jiabao Jin, Xiangyu Wang, Zhitao Shen, Wei Jia, Wentao Li, and Wei Wang. 2024. Bridging Speed and Accuracy to Approximate K-Nearest Neighbor Search. *arXiv:2404.16322* (2024).
[114] Wen Yang, Tao Li, Gai Fang, and Hong Wei. 2020. PASE: PostgreSQL Ultra-High-Dimensional Approximate Nearest Neighbor Search Extension. In *Proceedings of the International Conference on Management of Data (SIGMOD)*. 2241–2253.
[115] Yuanhang Yu, Dong Wen, Ying Zhang, Lu Qin, Wenjie Zhang, and Xuemin Lin. 2022. GPU-accelerated proximity graph approximate nearest Neighbor search and construction. In *IEEE International Conference on Data Engineering (ICDE)*. 552–564.
[116] Qiang Yue, Xiaoliang Xu, Yuxiang Wang, Yikun Tao, and Xuliyuan Luo. 2024. Routing-Guided Learned Product Quantization for Graph-Based Approximate Nearest Neighbor Search. In *IEEE 40th International Conference on Data Engineering (ICDE)*. 4870–4883.
[117] Shulin Zeng, Zhenhua Zhu, Jun Liu, Haoyu Zhang, Guohao Dai, Zixuan Zhou, Shuangchen Li, Xuefei Ning, Yuan Xie, Huazhong Yang, and Yu Wang. 2023. DF-GAS: a Distributed FPGA-as-a-Service Architecture towards Billion-Scale Graph-based Approximate Nearest Neighbor Search. In *Proceedings of the 56th Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*. 283–296.
[118] Jingtao Zhan, Jiaxin Mao, Yiqun Liu, Jiafeng Guo, Min Zhang, and Shaoping Ma. 2021. Jointly Optimizing Query Encoder and Product Quantization to Improve Retrieval Performance. In *The 30th ACM International Conference on Information and Knowledge Management (CIKM)*. 2487–2496.
[119] Haokui Zhang, Buzhou Tang, Wenze Hu, and Xiaoyu Wang. 2022. Connecting Compression Spaces with Transformer for Approximate Nearest Neighbor Search. In *European Conference on Computer Vision (ECCV)*, Vol. 13674. 515–530.
[120] Pengcheng Zhang, Bin Yao, Chao Gao, Bin Wu, Xiao He, Feifei Li, Yuanfei Lu, Chaoqun Zhan, and Feilong Tang. 2023. Learning-based query optimization for multi-probe approximate nearest neighbor search. *The VLDB Journal (VLDBJ)* 32, 3 (2023), 623–645.
[121] Qianxi Zhang, Shuotao Xu, Qi Chen, Guoxin Sui, Jiadong Xie, Zhizhen Cai, Yaoqi Chen, Yinxuan He, Yuqing Yang, Fan Yang, et al. 2023. VBASE: Unifying Online Vector Similarity Search and Relational Queries via Relaxed Monotonicity. In *17th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*. 377–395.
[122] Ting Zhang, Chao Du, and Jingdong Wang. 2014. Composite Quantization for Approximate Nearest Neighbor Search. In *Proceedings of the 31th International Conference on Machine Learning (ICML)*, Vol. 32. 838–846.
[123] Yunan Zhang, Shige Liu, and Jianguo Wang. 2024. Are There Fundamental Limitations in Supporting Vector Data Management in Relational Databases? A Case Study of PostgreSQL. In *40th IEEE International Conference on Data Engineering (ICDE)*. 3640–3653.
[124] Zili Zhang, Fangyue Liu, Gang Huang, Xuanzhe Liu, and Xin Jin. 2024. Fast Vector Query Processing for Large Datasets Beyond GPU Memory with Reordered Pipelining. In *21st USENIX Symposium on Networked Systems Design and Implementation (NSDI)*. 23–40.
[125] Weijie Zhao, Shulong Tan, and Ping Li. 2020. SONG: Approximate Nearest Neighbor Search on GPU. In *36th IEEE International Conference on Data Engineering (ICDE)*. 1033–1044.
[126] Xi Zhao, Yao Tian, Kai Huang, Bolong Zheng, and Xiaofang Zhou. 2023. Towards Efficient Index Construction and Approximate Nearest Neighbor Search in High-Dimensional Spaces. *Proceedings of the VLDB Endowment (PVLDB)* 16, 8 (2023), 1979–1991.
[127] Xi Zhao, Bolong Zheng, Xiaomeng Yi, Xiaofan Luan, Charles Xie, Xiaofang Zhou, and Christian S. Jensen. 2023. FARGO: Fast Maximum Inner Product Search via Global Multi-Probing. *Proceedings of the VLDB Endowment (PVLDB)* 16, 5 (2023), 1100–1112.
[128] Chaoji Zuo, Miao Qiao, Wenchao Zhou, Feifei Li, and Dong Deng. 2024. SeRF: Segment Graph for Range-Filtering Approximate Nearest Neighbor Search. *Proceedings of the ACM on Management of Data (PACMMOD)* 2, 1 (2024), 69:1–69:26.

Received October 2024; revised January 2025; accepted February 2025